2️⃣

Minimum Viable Product Deep Dive

Auto-Genie MVP - Notebook-Oriented Development Plan

Architecture Overview

The MVP will be built as a **modular Databricks notebook** with clear function boundaries. Each function will leverage Databricks system tables, Unity Catalog APIs, and Genie REST APIs.

Function Breakdown & Interrelationships

Stage 0: Configuration & Initialization

Function 1: `initialize_config()`

Functionality:

- Load workspace configuration
- Validate Databricks token and permissions
- Initialize SDK clients (Databricks SDK, REST API client)
- Set up logging and output paths

APIs/Tables Used:

- `WorkspaceClient()` from Databricks SDK
- Workspace API: `/api/2.0/workspace/get-status`
- Environment variables for tokens

Example Result:

```
{
  "workspace_url": "https://adb-123456789.azuredatabricks.net",
  "current_user": "sourav692@gmail.com",
  "catalog": "main",
  "schema": "sales",
  "genie_warehouse_id": "abc123def456",
  "status": "initialized"
}
```

Stage 1: Metadata Discovery & Profiling

Function 2: `extract_table_metadata(catalog, schema, table_names)`

Functionality:

- Extract schema, column names, data types, comments
- Get table properties (row count, size, last modified)
- Identify primary keys, foreign keys from constraints

System Tables Used:

- `system.information_schema.tables`
- `system.information_schema.columns`
- `system.information_schema.table_constraints`

- `system.information_schema.key_column_usage`

Example Result:

```
{
  "main.sales.orders": {
    "row_count": 1_250_000,
    "size_bytes": 450_000_000,
    "last_modified": "2026-02-15T10:30:00Z",
    "columns": [
      {
        "name": "order_id",
        "data_type": "BIGINT",
        "is_nullable": False,
        "comment": "Unique order identifier",
        "is_primary_key": True
      },
      {
        "name": "customer_id",
        "data_type": "BIGINT",
        "is_nullable": False,
        "comment": "Foreign key to customers table"
      },
      {
        "name": "order_date",
        "data_type": "DATE",
        "is_nullable": False,
        "comment": "Date order was placed"
      },
      {
        "name": "total_amount",
        "data_type": "DECIMAL(10,2)",
        "is_nullable": False,
        "comment": "Total order value in USD"
      }
    ]
  }
}
```

Function 3: `profile_column_statistics(catalog, schema, table_name)`

Functionality:

- Calculate distinct count, null percentage
- Identify low-cardinality columns (for entity matching)
- Detect PII patterns (email, phone, SSN regex)
- Sample top values for categorical columns

System Tables Used:

- `system.information_schema.column_statistics` (if available)
- Direct SQL queries using `spark.sql()`

Example Result:

```
{
  "main.sales.orders.customer_id": {
    "distinct_count": 85_000,
    "null_percentage": 0.02,
```

```
    "cardinality_category": "high",
    "recommend_entity_matching": False
  },
  "main.sales.orders.order_status": {
    "distinct_count": 5,
    "null_percentage": 0.0,
    "cardinality_category": "low",
    "recommend_entity_matching": True,
    "top_values": ["completed", "pending", "shipped", "cancelled", "refunded"]
  },
  "main.sales.customers.email": {
    "distinct_count": 84_500,
    "null_percentage": 0.01,
    "pii_detected": "email",
    "recommend_masking": True
  }
}
```

Stage 2: Relationship Discovery

Function 4: `discover_declared_relationships(catalog, schema)`

Functionality:

- Extract foreign key constraints from Unity Catalog
- Parse constraint metadata into join definitions

System Tables Used:

- `system.information_schema.table_constraints`
- `system.information_schema.constraint_column_usage`
- `system.information_schema.referential_constraints`

Example Result:

```
[
  {
    "source_table": "main.sales.orders",
    "source_column": "customer_id",
    "target_table": "main.sales.customers",
    "target_column": "customer_id",
    "confidence": 1.0,
    "method": "declared_fk"
  },
  {
    "source_table": "main.sales.order_items",
    "source_column": "order_id",
    "target_table": "main.sales.orders",
    "target_column": "order_id",
    "confidence": 1.0,
    "method": "declared_fk"
  }
]
```

Function 5: `discover_naming_pattern_relationships(table_metadata)`

Functionality:

- Identify column name patterns (`_id`, `_key`, `fk_*`)

- Match naming conventions across tables
- Score confidence based on data type alignment

APIs/Tables Used:

- Uses output from `extract_table_metadata()`
- Pattern matching algorithms (regex, fuzzy matching)

Example Result:

```
[
  {
    "source_table": "main.sales.order_items",
    "source_column": "product_id",
    "target_table": "main.sales.products",
    "target_column": "product_id",
    "confidence": 0.85,
    "method": "naming_pattern",
    "evidence": "Both INT64, suffix '_id', name match"
  },
  {
    "source_table": "main.sales.orders",
    "source_column": "warehouse_key",
    "target_table": "main.sales.warehouses",
    "target_column": "warehouse_id",
    "confidence": 0.70,
    "method": "naming_pattern",
    "evidence": "Suffix '_key' matches '_id', INT64 types"
  }
]
```

Function 6: `discover_query_pattern_relationships(catalog, schema, lookback_days=90)`

Functionality:

- Mine query history for JOIN patterns
- Extract table pairs frequently joined together
- Identify join columns from SQL AST parsing

System Tables Used:

- `system.query.history`
- `system.access.table_lineage` (if available)

Example Result:

```
[
  {
    "source_table": "main.sales.orders",
    "source_column": "customer_id",
    "target_table": "main.sales.customers",
    "target_column": "id",
    "confidence": 0.95,
    "method": "query_pattern",
    "query_count": 847,
    "users": 23,
    "sample_query": "SELECT o.*, c.name FROM orders o JOIN customers c ON o.customer_id =
c.id"
```

```
  }
]
```

Function 7: `merge_and_rank_relationships(declared, naming_pattern, query_pattern)`

Functionality:

- Combine all discovered relationships
- Remove duplicates and conflicting entries
- Rank by confidence score (declared > query_pattern > naming)
- Filter by threshold (default 0.75)

Example Result:

```
[
  {
    "rank": 1,
    "source": "main.sales.orders.customer_id",
    "target": "main.sales.customers.customer_id",
    "confidence": 1.0,
    "methods": ["declared_fk", "query_pattern", "naming_pattern"],
    "evidence_count": 3
  },
  {
    "rank": 2,
    "source": "main.sales.order_items.product_id",
    "target": "main.sales.products.product_id",
    "confidence": 0.88,
    "methods": ["query_pattern", "naming_pattern"],
    "evidence_count": 2
  }
]
```

Stage 3: Query Intelligence & Pattern Mining

Function 8: `extract_query_history(catalog, schema, lookback_days=90)`

Functionality:

- Pull all SQL queries touching target tables
- Filter out failed/cancelled queries
- Extract user, timestamp, execution time, result size

System Tables Used:

- `system.query.history`

Example Result:

```
[
  {
    "query_id": "01ef1234-5678-9abc",
    "query_text": "SELECT order_date, SUM(total_amount) FROM main.sales.orders GROUP BY or
der_date",
    "user": "data_analyst@company.com",
    "executed_at": "2026-02-10T14:22:00Z",
    "execution_time_ms": 3200,
    "rows_returned": 365,
    "tables_used": ["main.sales.orders"]
```

```
  },
  {
    "query_id": "01ef1234-5679-abcd",
    "query_text": "SELECT c.name, COUNT(*) as order_count FROM orders o JOIN customers c O
N o.customer_id = c.id GROUP BY c.name",
    "user": "business_user@company.com",
    "executed_at": "2026-02-12T09:15:00Z",
    "execution_time_ms": 5400,
    "rows_returned": 850,
    "tables_used": ["main.sales.orders", "main.sales.customers"]
  }
]
```

Function 9: `parse_and_cluster_queries(query_history)`

Functionality:

- Parse SQL into AST (Abstract Syntax Tree)
- Extract semantic patterns (aggregations, filters, joins)
- Cluster similar queries using TF-IDF + cosine similarity
- Identify top 20 most frequent patterns

Libraries Used:

- `sqlglot` for SQL parsing
- `sklearn.feature_extraction.text.TfidfVectorizer`
- `sklearn.cluster.DBSCAN`

Example Result:

```
{
  "clusters": [
    {
      "cluster_id": 0,
      "pattern": "daily_revenue_aggregation",
      "query_count": 127,
      "template": "SELECT order_date, SUM(total_amount) FROM orders GROUP BY order_date",
      "variations": [
        "WHERE order_date >= '2025-01-01'",
        "WHERE customer_id IN (...)",
        "ORDER BY order_date DESC"
      ]
    },
    {
      "cluster_id": 1,
      "pattern": "customer_order_join",
      "query_count": 98,
      "template": "SELECT c.*, COUNT(o.order_id) FROM customers c JOIN orders o ON c.id =
o.customer_id GROUP BY c.id",
      "common_columns": ["name", "email", "order_count"]
    }
  ]
}
```

Function 10: `generate_example_queries(query_clusters, top_n=10)`

Functionality:

- Convert query clusters to natural language descriptions
- Parameterize queries (replace literals with placeholders)
- Select most representative examples

Example Result:

```
[
  {
    "natural_language": "Show me daily revenue for the last 30 days",
    "sql": "SELECT order_date, SUM(total_amount) as revenue FROM main.sales.orders WHERE o
rder_date >= CURRENT_DATE - INTERVAL 30 DAYS GROUP BY order_date ORDER BY order_date",
    "frequency": 127,
    "avg_execution_time_ms": 3200
  },
  {
    "natural_language": "Which customers have placed more than 10 orders?",
    "sql": "SELECT c.name, c.email, COUNT(o.order_id) as order_count FROM main.sales.custo
mers c JOIN main.sales.orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id,
c.name, c.email HAVING COUNT(o.order_id) > 10",
    "frequency": 45,
    "avg_execution_time_ms": 4800
  }
]
```

Stage 4: Instruction Generation

Function 11: `generate_table_instructions(table_metadata, column_stats, relationships)`

Functionality:

- Create human-readable table descriptions
- Explain business context for each table
- Document key columns and their purpose
- Use LLM (optional) or template-based generation

APIs Used:

- Optionally: `openai.ChatCompletion` or Databricks Foundation Models API

Example Result:

```
{
  "main.sales.orders": {
    "description": "This table contains all customer orders placed through our e-commerce
platform since January 2020. Each row represents a single order with details about the cus
tomer, order date, and total amount. Use this table to analyze order trends, revenue patte
rns, and customer purchasing behavior.",
    "key_columns": {
      "order_id": "Unique identifier for each order. Use this to join with order_items tab
le.",
      "customer_id": "Links to the customers table. Use to get customer demographics.",
      "order_date": "Date the order was placed. Commonly filtered for time-based analysi
s.",
      "total_amount": "Total order value in USD including tax and shipping."
    },
    "common_use_cases": [
      "Daily/monthly revenue reporting",
      "Customer lifetime value calculation",
      "Order funnel analysis"
```

```
    ]
  }
}
```

Function 12: `generate_join_instructions(relationships)`

Functionality:

- Convert relationship list into Genie join definitions
- Add explanations for why tables should be joined
- Specify join type (INNER vs LEFT)

Example Result:

```
[
  {
    "join_definition": "main.sales.orders.customer_id = main.sales.customers.customer_id",
    "explanation": "Join orders with customers to get customer name, email, and demographi
c information. Use INNER JOIN when you only want orders with valid customer records, or LE
FT JOIN to include all orders even if customer data is missing.",
    "recommended_join_type": "INNER"
  },
  {
    "join_definition": "main.sales.order_items.order_id = main.sales.orders.order_id",
    "explanation": "Join order_items with orders to see the individual products in each or
der. Always use INNER JOIN since every order_item must have a parent order.",
    "recommended_join_type": "INNER"
  }
]
```

Function 13: `generate_sql_expressions(table_metadata, query_patterns)`

Functionality:

- Auto-create common measures (SUM, AVG, COUNT)
- Generate filters based on query patterns
- Create date dimensions (year, month, quarter)

Example Result:

```
{
  "measures": [
    {
      "name": "Total Revenue",
      "expression": "SUM(total_amount)",
      "description": "Sum of all order amounts"
    },
    {
      "name": "Average Order Value",
      "expression": "AVG(total_amount)",
      "description": "Average value per order"
    },
    {
      "name": "Order Count",
      "expression": "COUNT(order_id)",
      "description": "Total number of orders"
    }
  ],
```

```
  "filters": [
    {
      "name": "Recent Orders",
      "expression": "order_date >= CURRENT_DATE - INTERVAL 30 DAYS",
      "description": "Orders from the last 30 days"
    },
    {
      "name": "High Value Orders",
      "expression": "total_amount > 1000",
      "description": "Orders exceeding $1,000"
    }
  ],
  "dimensions": [
    {
      "name": "Order Year",
      "expression": "YEAR(order_date)",
      "description": "Year the order was placed"
    },
    {
      "name": "Order Month",
      "expression": "DATE_TRUNC('MONTH', order_date)",
      "description": "Month the order was placed"
    }
  ]
}
```

Stage 5: Knowledge Store Assembly

Function 14: `assemble_knowledge_store(table_metadata, relationships, instructions, example_queries, sql_expressions)`

Functionality:

- Combine all generated artifacts
- Structure into Genie API-compatible JSON format
- Validate completeness and consistency

Example Result:

```
{
  "space_name": "Sales Analytics Space",
  "tables": [
    {
      "catalog": "main",
      "schema": "sales",
      "table": "orders",
      "description": "Customer orders with dates and amounts...",
      "columns": [...],
      "sample_queries": [...]
    }
  ],
  "joins": [
    {
      "left_table": "main.sales.orders",
      "left_column": "customer_id",
      "right_table": "main.sales.customers",
      "right_column": "customer_id",
      "join_type": "INNER",
```

```
      "explanation": "..."
    }
  ],
  "sql_expressions": {
    "measures": [...],
    "filters": [...],
    "dimensions": [...]
  },
  "example_queries": [...],
  "instructions": "This space contains sales data from 2020-present..."
}
```

Stage 6: Validation

Function 15: `validate_knowledge_store(knowledge_store)`

Functionality:

- Check JSON structure completeness
- Validate SQL syntax for joins and expressions
- Verify table/column existence in Unity Catalog
- Test example queries against actual data

APIs Used:

- Databricks SQL Execution API
- `spark.sql()` for query validation

Example Result:

```
{
  "validation_status": "passed",
  "checks": [
    {
      "check": "structure_validation",
      "status": "passed",
      "details": "All required fields present"
    },
    {
      "check": "sql_syntax",
      "status": "passed",
      "details": "All 15 SQL expressions are valid"
    },
    {
      "check": "table_existence",
      "status": "passed",
      "details": "All 5 tables exist in Unity Catalog"
    },
    {
      "check": "example_queries",
      "status": "warning",
      "details": "8/10 queries executed successfully, 2 timed out (>30s)"
    }
  ],
  "warnings": [
    "Query 'Top 100 customers by lifetime value' took 42 seconds - consider optimization"
  ],
```

```
  "errors": []
}
```

Stage 7: Genie Space Deployment

Function 16: `create_genie_space(space_name, warehouse_id, description)`

Functionality:

- Call Genie API to create a new space
- Return space ID and URL

Genie API Used:

```
POST /api/2.0/genie/spaces
```

Example Result:

```
{
  "space_id": "01ef5678-90ab-cdef",
  "space_name": "Sales Analytics Space",
  "space_url": "https://adb-123456789.azuredatabricks.net/genie/spaces/01ef5678-90ab-cde
f",
  "warehouse_id": "abc123def456",
  "status": "created"
}
```

Function 17: `add_tables_to_space(space_id, tables)`

Functionality:

- Add Unity Catalog tables to Genie space
- Assign descriptions and column metadata

Genie API Used:

```
POST /api/2.0/genie/spaces/{space_id}/tables
```

Example Result:

```
{
  "space_id": "01ef5678-90ab-cdef",
  "tables_added": [
    "main.sales.orders",
    "main.sales.customers",
    "main.sales.products",
    "main.sales.order_items"
  ],
  "status": "success"
}
```

Function 18: `add_joins_to_space(space_id, joins)`

Functionality:

- Configure table relationships in Genie
- Add join explanations

Genie API Used:

```
POST /api/2.0/genie/spaces/{space_id}/instructions
```

Example Result:

```
{
  "space_id": "01ef5678-90ab-cdef",
  "joins_configured": 4,
  "status": "success"
}
```

Function 19: `add_sql_expressions_to_space(space_id, expressions)`

Functionality:

- Add measures, filters, dimensions
- Configure calculation logic

Genie API Used:

```
POST /api/2.0/genie/spaces/{space_id}/instructions
```

Example Result:

```
{
  "space_id": "01ef5678-90ab-cdef",
  "measures_added": 5,
  "filters_added": 3,
  "dimensions_added": 4,
  "status": "success"
}
```

Function 20: `add_example_queries_to_space(space_id, examples)`

Functionality:

- Upload natural language + SQL pairs
- Train Genie on common patterns

Genie API Used:

```
POST /api/2.0/genie/spaces/{space_id}/conversations
```

Example Result:

```
{
  "space_id": "01ef5678-90ab-cdef",
  "examples_added": 10,
  "status": "success"
}
```

Function 21: `finalize_space_deployment(space_id)`

Functionality:

- Set space status to “active”
- Run final validation
- Generate shareable link

Example Result:

```
{
  "space_id": "01ef5678-90ab-cdef",
  "status": "active",
  "deployment_time": "2026-02-21T15:45:30Z",
  "share_url": "https://adb-123456789.azuredatabricks.net/genie/spaces/01ef5678-90ab-cde
f",
  "validation_score": 0.82,
  "ready_for_use": True
}
```

Function Dependency Graph

```
initialize_config()
    ↓
extract_table_metadata() ──┬──→ profile_column_statistics()
                           │
                           ├──→ discover_declared_relationships()
                           │
                           ├──→ discover_naming_pattern_relationships()
                           │
                           └──→ extract_query_history()
                                    ↓
                                discover_query_pattern_relationships()
                                    ↓
                                merge_and_rank_relationships()
                                    ↓
extract_query_history() ──→ parse_and_cluster_queries()
                                    ↓
                                generate_example_queries()
                                    ↓
[All above outputs] ──→ generate_table_instructions()
                    ├──→ generate_join_instructions()
                    └──→ generate_sql_expressions()
                                    ↓
                        assemble_knowledge_store()
                                    ↓
                        validate_knowledge_store()
                                    ↓
                        create_genie_space()
                                    ↓
                        add_tables_to_space()
                                    ↓
                        add_joins_to_space()
                                    ↓
                        add_sql_expressions_to_space()
                                    ↓
                        add_example_queries_to_space()
                                    ↓
                        finalize_space_deployment()
```

Notebook Organization Structure

```
# Notebook 1: Configuration & Setup
- Cell 1: initialize_config()
- Cell 2: Test connection

# Notebook 2: Metadata Discovery
- Cell 1: extract_table_metadata()
- Cell 2: profile_column_statistics()
- Cell 3: Visualize results

# Notebook 3: Relationship Discovery
- Cell 1: discover_declared_relationships()
- Cell 2: discover_naming_pattern_relationships()
- Cell 3: discover_query_pattern_relationships()
- Cell 4: merge_and_rank_relationships()

# Notebook 4: Query Intelligence
- Cell 1: extract_query_history()
- Cell 2: parse_and_cluster_queries()
- Cell 3: generate_example_queries()

# Notebook 5: Instruction Generation
- Cell 1: generate_table_instructions()
- Cell 2: generate_join_instructions()
- Cell 3: generate_sql_expressions()

# Notebook 6: Assembly & Validation
- Cell 1: assemble_knowledge_store()
- Cell 2: validate_knowledge_store()
- Cell 3: Review & adjust

# Notebook 7: Deployment
- Cell 1: create_genie_space()
- Cell 2: add_tables_to_space()
- Cell 3: add_joins_to_space()
- Cell 4: add_sql_expressions_to_space()
- Cell 5: add_example_queries_to_space()
- Cell 6: finalize_space_deployment()
- Cell 7: Test deployed space
```

Key System Tables Summary

System Table	Used By Functions	Purpose
`system.information_schema.tables`	#2	Table metadata (row count, size)
`system.information_schema.columns`	#2, #3	Column details (type, nullability)
`system.information_schema.table_constraints`	#2, #4	Primary/foreign key constraints
`system.information_schema.referential_constraints`	#4	FK relationships
`system.information_schema.column_statistics`	#3	Column profiling stats
`system.query.history`	#8, #6	Query logs for pattern mining
`system.access.table_lineage`	#6	Lineage-based relationships

Genie API Endpoints Summary

Endpoint	Used By Functions	Purpose
`POST /api/2.0/genie/spaces`	#16	Create new space

Endpoint	Used By Functions	Purpose
`POST /api/2.0/genie/spaces/{id}/tables`	#17	Add tables to space
`POST /api/2.0/genie/spaces/{id}/instructions`	#18, #19	Add joins, expressions
`POST /api/2.0/genie/spaces/{id}/conversations`	#20	Add example queries
`PATCH /api/2.0/genie/spaces/{id}`	#21	Update space status
`GET /api/2.0/genie/spaces/{id}`	#21	Get space details

Would you like me to:

1. **Create detailed pseudo-code** for any specific function?
2. **Design the notebook cell execution flow** with expected output displays?
3. **Build error-handling patterns** for each stage?
4. **Create a sample configuration YAML** for input parameters?

Auto-Genie MVP - Notebook Cell Execution Flow with Expected Outputs

Notebook 1: Configuration & Setup

Cell 1.1: Install Required Libraries

```
# Install dependencies
%pip install databricks-sdk sqlglot scikit-learn plotly pandas numpy --quiet
dbutils.library.restartPython()
```

Expected Output:

```
Installing collected packages: databricks-sdk, sqlglot, scikit-learn, plotly
Successfully installed databricks-sdk-0.18.0 sqlglot-20.9.0 scikit-learn-1.3.2 plotly-5.1
8.0
Python interpreter will be restarted.
```

Cell 1.2: Import Libraries

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.sql import StatementState
import requests
import json
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from datetime import datetime, timedelta
import sqlglot
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import DBSCAN
import re
from collections import Counter

print("✅ All libraries imported successfully")
```

Expected Output:

```
✅ All libraries imported successfully
```

Cell 1.3: Configuration Setup

```
# Configuration
config = {
    "workspace_url": spark.conf.get("spark.databricks.workspaceUrl"),
    "catalog": "main",
    "schema": "sales",
    "tables": ["orders", "customers", "products", "order_items", "warehouses"],
    "genie_warehouse_id": "abc123def456",  # Replace with your SQL warehouse ID
    "lookback_days": 90,
    "confidence_threshold": 0.75,
    "output_path": "/dbfs/tmp/auto_genie_outputs"
}

# Initialize Databricks SDK
w = WorkspaceClient()

# Display configuration
print("="*60)
print("AUTO-GENIE CONFIGURATION")
print("="*60)
for key, value in config.items():
    print(f"{key:25s}: {value}")
print("="*60)
```

Expected Output:

```
============================================================
AUTO-GENIE CONFIGURATION
============================================================
workspace_url             : adb-123456789.12.azuredatabricks.net
catalog                   : main
schema                    : sales
tables                    : ['orders', 'customers', 'products', 'order_items', 'warehouse
s']
genie_warehouse_id        : abc123def456
lookback_days             : 90
confidence_threshold      : 0.75
output_path               : /dbfs/tmp/auto_genie_outputs
============================================================
```

Cell 1.4: Test Database Connectivity

```
# Test connection to Unity Catalog
try:
    test_query = f"""
    SELECT COUNT(*) as table_count
    FROM system.information_schema.tables
    WHERE table_catalog = '{config['catalog']}'
    AND table_schema = '{config['schema']}'
    """
    result = spark.sql(test_query).collect()[0]
```

```
    print(f"✅ Successfully connected to Unity Catalog")
    print(f"✅ Found {result['table_count']} tables in {config['catalog']}.{config['schem
a']}")

    # Test Genie API access
    current_user = w.current_user.me()
    print(f"✅ Authenticated as: {current_user.user_name}")
    print(f"✅ User ID: {current_user.id}")

except Exception as e:
    print(f"❌ Connection failed: {str(e)}")
```

Expected Output:

```
✅ Successfully connected to Unity Catalog
✅ Found 5 tables in main.sales
✅ Authenticated as: sourav692@gmail.com
✅ User ID: 1234567890123456
```

Notebook 2: Metadata Discovery & Profiling

Cell 2.1: Extract Table Metadata Function

```
def extract_table_metadata(catalog, schema, table_names):
    """Extract comprehensive metadata for specified tables"""

    metadata = {}

    for table_name in table_names:
        full_table_name = f"{catalog}.{schema}.{table_name}"

        # Get table-level metadata
        table_info_query = f"""
        SELECT
            table_catalog,
            table_schema,
            table_name,
            table_type,
            comment
        FROM system.information_schema.tables
        WHERE table_catalog = '{catalog}'
        AND table_schema = '{schema}'
        AND table_name = '{table_name}'
        """
        table_info = spark.sql(table_info_query).collect()

        if not table_info:
            print(f"⚠  Table {full_table_name} not found")
            continue

        # Get column metadata
        columns_query = f"""
        SELECT
            column_name,
            data_type,
            is_nullable,
            column_default,
```

```
            comment
        FROM system.information_schema.columns
        WHERE table_catalog = '{catalog}'
        AND table_schema = '{schema}'
        AND table_name = '{table_name}'
        ORDER BY ordinal_position
        """
        columns = spark.sql(columns_query).collect()

        # Get table statistics
        stats_query = f"DESCRIBE DETAIL {full_table_name}"
        stats = spark.sql(stats_query).collect()[0]

        # Get constraints
        constraints_query = f"""
        SELECT
            constraint_name,
            constraint_type
        FROM system.information_schema.table_constraints
        WHERE table_catalog = '{catalog}'
        AND table_schema = '{schema}'
        AND table_name = '{table_name}'
        """
        constraints = spark.sql(constraints_query).collect()

        metadata[full_table_name] = {
            "table_info": table_info[0].asDict(),
            "columns": [col.asDict() for col in columns],
            "row_count": stats['numRows'] if stats['numRows'] else 0,
            "size_bytes": stats['sizeInBytes'],
            "last_modified": stats['lastModified'],
            "constraints": [c.asDict() for c in constraints]
        }

        print(f"✅ Extracted metadata for {full_table_name}")

    return metadata

# Execute extraction
print("="*60)
print("EXTRACTING TABLE METADATA")
print("="*60)
table_metadata = extract_table_metadata(
    config['catalog'],
    config['schema'],
    config['tables']
)
print(f"\n✅ Metadata extraction complete for {len(table_metadata)} tables")
```

Expected Output:

```
============================================================
EXTRACTING TABLE METADATA
============================================================
✅ Extracted metadata for main.sales.orders
✅ Extracted metadata for main.sales.customers
✅ Extracted metadata for main.sales.products
✅ Extracted metadata for main.sales.order_items
```

```
✅ Extracted metadata for main.sales.warehouses

✅ Metadata extraction complete for 5 tables
```

Cell 2.2: Display Metadata Summary

```
# Create summary DataFrame
summary_data = []
for table_name, metadata in table_metadata.items():
    summary_data.append({
        "Table": table_name.split('.')[-1],
        "Row Count": f"{metadata['row_count']:,}",
        "Size (MB)": f"{metadata['size_bytes'] / (1024*1024):.2f}",
        "Columns": len(metadata['columns']),
        "Constraints": len(metadata['constraints']),
        "Last Modified": metadata['last_modified'].strftime("%Y-%m-%d %H:%M")
    })

summary_df = pd.DataFrame(summary_data)

print("="*80)
print("TABLE METADATA SUMMARY")
print("="*80)
display(summary_df)

# Visualize table sizes
fig = px.bar(
    summary_df,
    x='Table',
    y=[float(s) for s in summary_df['Size (MB)']],
    title='Table Sizes (MB)',
    labels={'y': 'Size (MB)'},
    color='Table'
)
fig.show()
```

Expected Output:

```
================================================================================
TABLE METADATA SUMMARY
================================================================================
```

Table	Row Count	Size (MB)	Columns	Constraints	Last Modified
orders	1,250,000	429.34	8	2	2026-02-15 10:30
customers	85,000	45.67	12	1	2026-02-14 08:15
products	5,432	2.18	10	1	2026-01-20 14:22
order_items	3,890,000	876.54	6	2	2026-02-15 10:31
warehouses	12	0.01	7	1	2025-11-10 09:00

[Interactive bar chart showing table sizes]

Cell 2.3: Profile Column Statistics

```
def profile_column_statistics(catalog, schema, table_name):
    """Profile column-level statistics"""
```

```
    full_table_name = f"{catalog}.{schema}.{table_name}"
    columns = table_metadata[full_table_name]['columns']

    stats = []

    for col in columns:
        col_name = col['column_name']
        col_type = col['data_type']

        # Calculate distinct count and nulls
        stats_query = f"""
        SELECT
            COUNT(DISTINCT {col_name}) as distinct_count,
            COUNT(*) as total_count,
            SUM(CASE WHEN {col_name} IS NULL THEN 1 ELSE 0 END) as null_count
        FROM {full_table_name}
        """

        try:
            result = spark.sql(stats_query).collect()[0]

            null_pct = (result['null_count'] / result['total_count'] * 100) if result['tot
al_count'] > 0 else 0
            cardinality = result['distinct_count']

            # Categorize cardinality
            if cardinality <= 10:
                cardinality_cat = "very_low"
                entity_matching = True
            elif cardinality <= 100:
                cardinality_cat = "low"
                entity_matching = True
            elif cardinality <= 1000:
                cardinality_cat = "medium"
                entity_matching = True
            else:
                cardinality_cat = "high"
                entity_matching = False

            # Detect PII patterns
            pii_type = None
            if re.search(r'email', col_name, re.IGNORECASE):
                pii_type = "email"
            elif re.search(r'phone|mobile', col_name, re.IGNORECASE):
                pii_type = "phone"
            elif re.search(r'ssn|social', col_name, re.IGNORECASE):
                pii_type = "ssn"

            stats.append({
                "column": col_name,
                "data_type": col_type,
                "distinct_count": cardinality,
                "null_percentage": round(null_pct, 2),
                "cardinality_category": cardinality_cat,
                "recommend_entity_matching": entity_matching,
                "pii_detected": pii_type,
                "recommend_masking": pii_type is not None
            })
```

```
        except Exception as e:
            print(f"⚠  Could not profile {col_name}: {str(e)}")

    return stats

# Profile all tables
print("="*60)
print("PROFILING COLUMN STATISTICS")
print("="*60)

column_profiles = {}
for table_name in config['tables']:
    full_name = f"{config['catalog']}.{config['schema']}.{table_name}"
    print(f"Profiling {table_name}...", end=" ")
    column_profiles[full_name] = profile_column_statistics(
        config['catalog'],
        config['schema'],
        table_name
    )
    print(f"✅ {len(column_profiles[full_name])} columns profiled")

print(f"\n✅ Column profiling complete")
```

Expected Output:

```
============================================================
PROFILING COLUMN STATISTICS
============================================================
Profiling orders... ✅ 8 columns profiled
Profiling customers... ✅ 12 columns profiled
Profiling products... ✅ 10 columns profiled
Profiling order_items... ✅ 6 columns profiled
Profiling warehouses... ✅ 7 columns profiled

✅ Column profiling complete
```

Cell 2.4: Display Column Profile for Orders Table

```
# Display detailed profile for orders table
orders_profile = column_profiles[f"{config['catalog']}.{config['schema']}.orders"]
profile_df = pd.DataFrame(orders_profile)

print("="*80)
print("COLUMN PROFILE: main.sales.orders")
print("="*80)
display(profile_df[['column', 'data_type', 'distinct_count', 'null_percentage',
                    'cardinality_category', 'recommend_entity_matching', 'pii_detected']])

# Highlight key findings
print("\n📊 KEY FINDINGS:")
entity_match_cols = [c['column'] for c in orders_profile if c['recommend_entity_matchin
g']]
pii_cols = [c['column'] for c in orders_profile if c['pii_detected']]

print(f"  • {len(entity_match_cols)} columns recommended for entity matching: {', '.join(e
ntity_match_cols)}")
```

```
print(f"  • {len(pii_cols)} columns contain PII: {', '.join(pii_cols) if pii_cols else 'No
ne'}")
```

Expected Output:

```
================================================================================
COLUMN PROFILE: main.sales.orders
================================================================================
```

column	data_type	distinct_count	null_percentage	cardinality_category	recommend_entity_matching	pii_det
order_id	BIGINT	1,250,000	0.00	high	False	None
customer_id	BIGINT	85,000	0.02	high	False	None
order_date	DATE	1,095	0.00	medium	True	None
order_status	STRING	5	0.00	very_low	True	None
total_amount	DECIMAL(10,2)	45,678	0.00	high	False	None
payment_method	STRING	4	0.01	very_low	True	None
shipping_address	STRING	78,234	1.23	high	False	None
created_at	TIMESTAMP	1,249,876	0.00	high	False	None

```
📊 KEY FINDINGS:
  • 3 columns recommended for entity matching: order_date, order_status, payment_method
  • 0 columns contain PII: None
```

Notebook 3: Relationship Discovery

Cell 3.1: Discover Declared Relationships

```
def discover_declared_relationships(catalog, schema):
    """Extract foreign key relationships from Unity Catalog"""

    relationships = []

    # Query for referential constraints
    fk_query = f"""
    SELECT
        tc.table_name as source_table,
        kcu.column_name as source_column,
        rc.referenced_table_name as target_table,
        rc.referenced_column_name as target_column
    FROM system.information_schema.table_constraints tc
    JOIN system.information_schema.key_column_usage kcu
        ON tc.constraint_name = kcu.constraint_name
        AND tc.table_catalog = kcu.table_catalog
        AND tc.table_schema = kcu.table_schema
        AND tc.table_name = kcu.table_name
    JOIN (
        SELECT
            constraint_name,
            table_name as referenced_table_name,
            column_name as referenced_column_name
        FROM system.information_schema.key_column_usage
        WHERE table_catalog = '{catalog}'
        AND table_schema = '{schema}'
    ) rc ON tc.constraint_name = rc.constraint_name
    WHERE tc.table_catalog = '{catalog}'
```

```
    AND tc.table_schema = '{schema}'
    AND tc.constraint_type = 'FOREIGN KEY'
    """

    try:
        results = spark.sql(fk_query).collect()

        for row in results:
            relationships.append({
                "source_table": f"{catalog}.{schema}.{row['source_table']}",
                "source_column": row['source_column'],
                "target_table": f"{catalog}.{schema}.{row['target_table']}",
                "target_column": row['target_column'],
                "confidence": 1.0,
                "method": "declared_fk"
            })

    except Exception as e:
        print(f"⚠  No declared foreign keys found or error: {str(e)}")

    return relationships

# Execute discovery
print("="*60)
print("DISCOVERING DECLARED RELATIONSHIPS")
print("="*60)

declared_relationships = discover_declared_relationships(
    config['catalog'],
    config['schema']
)

if declared_relationships:
    for rel in declared_relationships:
        print(f"✅ {rel['source_table']}.{rel['source_column']} → {rel['target_table']}.{r
el['target_column']}")
else:
    print("ℹ️  No declared foreign key relationships found")

print(f"\n📊 Found {len(declared_relationships)} declared relationships")
```

Expected Output:

```
============================================================
DISCOVERING DECLARED RELATIONSHIPS
============================================================
✅ main.sales.orders.customer_id → main.sales.customers.customer_id
✅ main.sales.order_items.order_id → main.sales.orders.order_id
✅ main.sales.order_items.product_id → main.sales.products.product_id
✅ main.sales.orders.warehouse_id → main.sales.warehouses.warehouse_id

📊 Found 4 declared relationships
```

Cell 3.2: Discover Naming Pattern Relationships

```
def discover_naming_pattern_relationships(table_metadata):
    """Discover relationships based on column naming patterns"""
```

```
    relationships = []

    # Extract all columns from all tables
    all_columns = {}
    for table_name, metadata in table_metadata.items():
        all_columns[table_name] = [
            (col['column_name'], col['data_type'])
            for col in metadata['columns']
        ]

    # Pattern matching rules
    id_patterns = [r'(.+)_id$', r'(.+)_key$', r'^fk_(.+)$']

    for source_table, source_cols in all_columns.items():
        for source_col, source_type in source_cols:

            # Check if column matches ID pattern
            for pattern in id_patterns:
                match = re.search(pattern, source_col, re.IGNORECASE)
                if match:
                    entity_name = match.group(1)

                    # Look for matching table and column
                    for target_table, target_cols in all_columns.items():
                        if source_table == target_table:
                            continue

                        # Check if table name contains entity name
                        target_table_simple = target_table.split('.')[-1]

                        if entity_name in target_table_simple or target_table_simple in en
tity_name:
                            # Look for matching column in target table
                            for target_col, target_type in target_cols:
                                # Check for ID/key columns with matching types
                                if (target_col.endswith('_id') or target_col.endswith('_ke
y') or target_col == 'id') \
                                   and source_type == target_type:

                                    confidence = 0.85 if source_col == target_col else 0.7
0

                                    relationships.append({
                                        "source_table": source_table,
                                        "source_column": source_col,
                                        "target_table": target_table,
                                        "target_column": target_col,
                                        "confidence": confidence,
                                        "method": "naming_pattern",
                                        "evidence": f"Pattern '{pattern}', types match ({s
ource_type})"
                                    })

    return relationships

# Execute discovery
print("="*60)
```

```
print("DISCOVERING NAMING PATTERN RELATIONSHIPS")
print("="*60)

naming_relationships = discover_naming_pattern_relationships(table_metadata)

for rel in naming_relationships[:10]:  # Show first 10
    print(f"✅ {rel['source_table'].split('.')[-1]}.{rel['source_column']} → "
          f"{rel['target_table'].split('.')[-1]}.{rel['target_column']} "
          f"(confidence: {rel['confidence']:.2f})")

print(f"\n📊 Found {len(naming_relationships)} naming pattern relationships")
```

Expected Output:

```
============================================================
DISCOVERING NAMING PATTERN RELATIONSHIPS
============================================================
✅ orders.customer_id → customers.customer_id (confidence: 0.85)
✅ orders.warehouse_id → warehouses.warehouse_id (confidence: 0.85)
✅ order_items.order_id → orders.order_id (confidence: 0.85)
✅ order_items.product_id → products.product_id (confidence: 0.85)
✅ orders.customer_id → customers.id (confidence: 0.70)

📊 Found 5 naming pattern relationships
```

Cell 3.3: Discover Query Pattern Relationships

```
def discover_query_pattern_relationships(catalog, schema, lookback_days=90):
    """Mine query history for JOIN patterns"""

    relationships = []

    # Calculate date range
    end_date = datetime.now()
    start_date = end_date - timedelta(days=lookback_days)

    # Query history
    history_query = f"""
    SELECT
        statement_text,
        user_name,
        executed_as_user_name,
        start_time,
        total_duration_ms
    FROM system.query.history
    WHERE start_time >= '{start_date.strftime("%Y-%m-%d")}'
    AND statement_type = 'SELECT'
    AND error_message IS NULL
    AND LOWER(statement_text) LIKE '%{schema}%'
    AND LOWER(statement_text) LIKE '%join%'
    ORDER BY start_time DESC
    LIMIT 5000
    """

    try:
        queries = spark.sql(history_query).collect()
        print(f"ℹ️  Analyzing {len(queries)} queries with JOINs...")
```

```
        # Parse JOIN patterns
        join_patterns = Counter()

        for query in queries:
            sql_text = query['statement_text'].lower()

            # Extract JOIN patterns using regex
            join_pattern = r'(\w+)\s+(?:inner\s+|left\s+|right\s+)?join\s+(\w+)\s+(?:as\s+
\w+\s+)?on\s+(\w+)\.(\w+)\s*=\s*(\w+)\.(\w+)'
            matches = re.findall(join_pattern, sql_text)

            for match in matches:
                # match: (table1, table2, alias1, col1, alias2, col2)
                if len(match) == 6:
                    # Create a normalized join signature
                    tables = tuple(sorted([match[0], match[1]]))
                    columns = (match[3], match[5])
                    join_sig = (tables, columns)
                    join_patterns[join_sig] += 1

        # Convert patterns to relationships
        for (tables, columns), count in join_patterns.items():
            if count >= 3:  # Minimum 3 occurrences
                # Calculate confidence based on frequency
                confidence = min(0.95, 0.60 + (count / 100))

                relationships.append({
                    "source_table": f"{catalog}.{schema}.{tables[0]}",
                    "source_column": columns[0],
                    "target_table": f"{catalog}.{schema}.{tables[1]}",
                    "target_column": columns[1],
                    "confidence": round(confidence, 2),
                    "method": "query_pattern",
                    "query_count": count,
                    "evidence": f"Found in {count} queries"
                })

    except Exception as e:
        print(f"⚠  Error analyzing query history: {str(e)}")

    return relationships

# Execute discovery
print("="*60)
print("DISCOVERING QUERY PATTERN RELATIONSHIPS")
print("="*60)

query_relationships = discover_query_pattern_relationships(
    config['catalog'],
    config['schema'],
    config['lookback_days']
)

for rel in query_relationships:
    print(f"✅ {rel['source_table'].split('.')[-1]}.{rel['source_column']} → "
          f"{rel['target_table'].split('.')[-1]}.{rel['target_column']} "
          f"(confidence: {rel['confidence']:.2f}, queries: {rel['query_count']})")
```

```
print(f"\n📊 Found {len(query_relationships)} query pattern relationships")
```

Expected Output:

```
============================================================
DISCOVERING QUERY PATTERN RELATIONSHIPS
============================================================
ℹ️  Analyzing 847 queries with JOINs...
✅ orders.customer_id → customers.customer_id (confidence: 0.95, queries: 234)
✅ order_items.order_id → orders.order_id (confidence: 0.92, queries: 189)
✅ order_items.product_id → products.product_id (confidence: 0.88, queries: 156)
✅ orders.warehouse_id → warehouses.warehouse_id (confidence: 0.75, queries: 67)
✅ customers.id → orders.customer_id (confidence: 0.71, queries: 45)

📊 Found 5 query pattern relationships
```

Cell 3.4: Merge and Rank All Relationships

```
def merge_and_rank_relationships(declared, naming, query_patterns, threshold=0.75):
    """Merge all discovered relationships and rank by confidence"""

    # Combine all relationships
    all_relationships = {}

    # Add declared relationships (highest priority)
    for rel in declared:
        key = f"{rel['source_table']}.{rel['source_column']}->{rel['target_table']}.{rel
['target_column']}"
        all_relationships[key] = rel.copy()
        all_relationships[key]['methods'] = ['declared_fk']
        all_relationships[key]['evidence_count'] = 1

    # Add naming pattern relationships
    for rel in naming:
        key = f"{rel['source_table']}.{rel['source_column']}->{rel['target_table']}.{rel
['target_column']}"
        if key in all_relationships:
            all_relationships[key]['methods'].append('naming_pattern')
            all_relationships[key]['evidence_count'] += 1
        else:
            all_relationships[key] = rel.copy()
            all_relationships[key]['methods'] = ['naming_pattern']
            all_relationships[key]['evidence_count'] = 1

    # Add query pattern relationships
    for rel in query_patterns:
        key = f"{rel['source_table']}.{rel['source_column']}->{rel['target_table']}.{rel
['target_column']}"
        if key in all_relationships:
            all_relationships[key]['methods'].append('query_pattern')
            all_relationships[key]['evidence_count'] += 1
            # Boost confidence if multiple methods agree
            all_relationships[key]['confidence'] = min(1.0, all_relationships[key]['confid
ence'] + 0.05)
            all_relationships[key]['query_count'] = rel.get('query_count', 0)
        else:
```

```
            all_relationships[key] = rel.copy()
            all_relationships[key]['methods'] = ['query_pattern']
            all_relationships[key]['evidence_count'] = 1

    # Convert to list and sort by confidence
    ranked = list(all_relationships.values())
    ranked.sort(key=lambda x: (x['evidence_count'], x['confidence']), reverse=True)

    # Filter by threshold
    filtered = [r for r in ranked if r['confidence'] >= threshold]

    # Add rank
    for i, rel in enumerate(filtered, 1):
        rel['rank'] = i

    return filtered

# Execute merge and ranking
print("="*60)
print("MERGING AND RANKING RELATIONSHIPS")
print("="*60)

final_relationships = merge_and_rank_relationships(
    declared_relationships,
    naming_relationships,
    query_relationships,
    config['confidence_threshold']
)

# Display results
print(f"\n📊 FINAL RELATIONSHIP SUMMARY")
print(f"   Total unique relationships: {len(final_relationships)}")
print(f"   Confidence threshold: {config['confidence_threshold']}")
print()

for rel in final_relationships:
    methods_str = ", ".join(rel['methods'])
    print(f"#{rel['rank']} [{rel['confidence']:.2f}] "
          f"{rel['source_table'].split('.')[-1]}.{rel['source_column']} → "
          f"{rel['target_table'].split('.')[-1]}.{rel['target_column']}")
    print(f"    Methods: {methods_str} | Evidence: {rel['evidence_count']} sources")
    print()
```

Expected Output:

```
============================================================
MERGING AND RANKING RELATIONSHIPS
============================================================

📊 FINAL RELATIONSHIP SUMMARY
   Total unique relationships: 5
   Confidence threshold: 0.75

#1 [1.00] orders.customer_id → customers.customer_id
    Methods: declared_fk, naming_pattern, query_pattern | Evidence: 3 sources

#2 [1.00] order_items.order_id → orders.order_id
    Methods: declared_fk, naming_pattern, query_pattern | Evidence: 3 sources
```

```
#3 [1.00] order_items.product_id → products.product_id
    Methods: declared_fk, naming_pattern, query_pattern | Evidence: 3 sources

#4 [0.95] orders.warehouse_id → warehouses.warehouse_id
    Methods: declared_fk, naming_pattern, query_pattern | Evidence: 3 sources

#5 [0.85] customers.customer_id → orders.customer_id
    Methods: naming_pattern, query_pattern | Evidence: 2 sources
```

Cell 3.5: Visualize Relationship Graph

```
# Create relationship graph visualization
import networkx as nx

# Build graph
G = nx.DiGraph()

for rel in final_relationships:
    source = rel['source_table'].split('.')[-1]
    target = rel['target_table'].split('.')[-1]
    G.add_edge(source, target,
               weight=rel['confidence'],
               label=f"{rel['source_column']}→{rel['target_column']}")

# Prepare data for Plotly
edge_trace = []
for edge in G.edges(data=True):
    edge_trace.append({
        'source': edge[0],
        'target': edge[1],
        'confidence': edge[2]['weight'],
        'label': edge[2]['label']
    })

# Create simple table visualization instead of network graph
edge_df = pd.DataFrame(edge_trace)

print("="*60)
print("RELATIONSHIP GRAPH")
print("="*60)
display(edge_df)

# Summary statistics
print(f"\n📊 GRAPH STATISTICS:")
print(f"   Nodes (tables): {G.number_of_nodes()}")
print(f"   Edges (relationships): {G.number_of_edges()}")
print(f"   Average confidence: {sum(e['confidence'] for e in edge_trace)/len(edge_trace):.
2f}")
```

Expected Output:

```
============================================================
RELATIONSHIP GRAPH
============================================================
```

source	target	confidence	label
orders	customers	1.00	customer_id → customer_id
order_items	orders	1.00	order_id → order_id
order_items	products	1.00	product_id → product_id
orders	warehouses	0.95	warehouse_id → warehouse_id
customers	orders	0.85	customer_id → customer_id

```
📊 GRAPH STATISTICS:
   Nodes (tables): 5
   Edges (relationships): 5
   Average confidence: 0.96
```

Notebook 4: Query Intelligence & Pattern Mining

Cell 4.1: Extract Query History

```
def extract_query_history(catalog, schema, lookback_days=90):
    """Extract comprehensive query history for target schema"""

    end_date = datetime.now()
    start_date = end_date - timedelta(days=lookback_days)

    query = f"""
    SELECT
        query_id,
        statement_text,
        user_name,
        executed_as_user_name,
        start_time,
        end_time,
        total_duration_ms,
        rows_produced,
        statement_type,
        warehouse_id
    FROM system.query.history
    WHERE start_time >= '{start_date.strftime("%Y-%m-%d")}'
    AND statement_type = 'SELECT'
    AND error_message IS NULL
    AND (
        LOWER(statement_text) LIKE '%{catalog}.{schema}%'
        OR LOWER(statement_text) LIKE '%{schema}.%'
    )
    ORDER BY start_time DESC
    LIMIT 10000
    """

    queries = spark.sql(query).collect()

    return [q.asDict() for q in queries]

# Execute extraction
print("="*60)
print("EXTRACTING QUERY HISTORY")
print("="*60)
print(f"Date range: {(datetime.now() - timedelta(days=config['lookback_days'])).strftime
```

```
('%Y-%m-%d')} to {datetime.now().strftime('%Y-%m-%d')}")
print()

query_history = extract_query_history(
    config['catalog'],
    config['schema'],
    config['lookback_days']
)

print(f"✅ Extracted {len(query_history)} queries")

# Display sample
if query_history:
    print("\n📋 SAMPLE QUERIES:")
    for i, q in enumerate(query_history[:3], 1):
        print(f"\n{i}. User: {q['user_name']} | Duration: {q['total_duration_ms']}ms")
        print(f"   {q['statement_text'][:150]}...")
```

Expected Output:

```
============================================================
EXTRACTING QUERY HISTORY
============================================================
Date range: 2025-11-23 to 2026-02-21

✅ Extracted 2,847 queries

📋 SAMPLE QUERIES:

1. User: data_analyst@company.com | Duration: 3240ms
   SELECT order_date, SUM(total_amount) as daily_revenue FROM main.sales.orders WHERE orde
r_date >= '2026-01-01' GROUP BY order_date ORDER BY order_date...

2. User: business_user@company.com | Duration: 5430ms
   SELECT c.name, c.email, COUNT(o.order_id) as order_count, SUM(o.total_amount) as lifeti
me_value FROM main.sales.customers c JOIN main.sales.orders...

3. User: report_scheduler@company.com | Duration: 12890ms
   SELECT p.product_name, p.category, SUM(oi.quantity) as units_sold, SUM(oi.quantity * o
i.unit_price) as revenue FROM main.sales.products p JOIN main...
```

Cell 4.2: Parse and Cluster Queries

```
def parse_and_cluster_queries(query_history, n_clusters=20):
    """Parse SQL and cluster similar queries"""

    from sqlglot import parse, exp

    parsed_queries = []

    print("Parsing SQL queries...")
    for i, query in enumerate(query_history):
        try:
            # Parse SQL
            statements = parse(query['statement_text'], read='databricks')

            if statements:
```

```
                stmt = statements[0]

                # Extract semantic features
                tables = [t.name for t in stmt.find_all(exp.Table)]
                columns = [c.name for c in stmt.find_all(exp.Column)]
                aggregations = [a.sql() for a in stmt.find_all(exp.AggFunc)]
                joins = len(list(stmt.find_all(exp.Join)))
                where_clauses = len(list(stmt.find_all(exp.Where)))
                group_by = len(list(stmt.find_all(exp.Group)))

                parsed_queries.append({
                    'query_id': query['query_id'],
                    'original_sql': query['statement_text'],
                    'tables': tables,
                    'columns': columns,
                    'aggregations': aggregations,
                    'joins': joins,
                    'where_clauses': where_clauses,
                    'group_by': group_by > 0,
                    'user': query['user_name'],
                    'duration_ms': query['total_duration_ms']
                })
        except Exception as e:
            continue

        if (i + 1) % 500 == 0:
            print(f"  Parsed {i + 1}/{len(query_history)} queries...")

    print(f"✅ Successfully parsed {len(parsed_queries)} queries\n")

    # Cluster queries based on semantic similarity
    print("Clustering queries...")

    # Create feature vectors
    query_features = []
    for q in parsed_queries:
        feature_text = (
            " ".join(q['tables']) + " " +
            " ".join(q['aggregations']) + " " +
            ("join" * q['joins']) + " " +
            ("where" * q['where_clauses']) + " " +
            ("groupby" if q['group_by'] else "")
        )
        query_features.append(feature_text)

    # TF-IDF vectorization
    vectorizer = TfidfVectorizer(max_features=50, stop_words='english')
    X = vectorizer.fit_transform(query_features)

    # Clustering
    clustering = DBSCAN(eps=0.4, min_samples=2, metric='cosine')
    clusters = clustering.fit_predict(X.toarray())

    # Group queries by cluster
    clustered_queries = {}
    for i, cluster_id in enumerate(clusters):
        if cluster_id == -1:  # Noise
            continue
```

```
        if cluster_id not in clustered_queries:
            clustered_queries[cluster_id] = []
        clustered_queries[cluster_id].append(parsed_queries[i])

    print(f"✅ Identified {len(clustered_queries)} query patterns\n")

    return parsed_queries, clustered_queries

# Execute parsing and clustering
print("="*60)
print("PARSING AND CLUSTERING QUERIES")
print("="*60)

parsed_queries, clustered_queries = parse_and_cluster_queries(query_history)

# Display cluster summary
print("📊 CLUSTER SUMMARY:")
print(f"{'Cluster':<10} {'Size':<10} {'Avg Duration (ms)':<20} {'Common Tables'}")
print("-" * 80)

for cluster_id, queries in sorted(clustered_queries.items(), key=lambda x: len(x[1]), reve
rse=True)[:10]:
    avg_duration = sum(q['duration_ms'] for q in queries) / len(queries)
    all_tables = []
    for q in queries:
        all_tables.extend(q['tables'])
    common_tables = Counter(all_tables).most_common(2)
    table_str = ", ".join([f"{t[0]}({t[1]})" for t in common_tables])

    print(f"{cluster_id:<10} {len(queries):<10} {avg_duration:<20.0f} {table_str}")
```

Expected Output:

```
============================================================
PARSING AND CLUSTERING QUERIES
============================================================
Parsing SQL queries...
  Parsed 500/2847 queries...
  Parsed 1000/2847 queries...
  Parsed 1500/2847 queries...
  Parsed 2000/2847 queries...
  Parsed 2500/2847 queries...
✅ Successfully parsed 2734 queries

Clustering queries...
✅ Identified 18 query patterns

📊 CLUSTER SUMMARY:
Cluster    Size       Avg Duration (ms)    Common Tables
--------------------------------------------------------------------------------
0          234        3420                 orders(234), customers(89)
1          189        4850                 order_items(189), orders(189)
2          156        5230                 order_items(156), products(156)
3          127        2890                 orders(127)
4          98         6740                 customers(98), orders(98)
5          67         4120                 orders(67), warehouses(67)
6          54         3560                 products(54)
7          45         7890                 customers(45), orders(45)
```

```
8          38         5240                 order_items(38), products(38)
9          32         4670                 orders(32), customers(32)
```

Cell 4.3: Generate Example Queries

```
def generate_example_queries(clustered_queries, parsed_queries, top_n=10):
    """Generate representative example queries with NL descriptions"""

    examples = []

    # Sort clusters by size
    sorted_clusters = sorted(clustered_queries.items(),
                             key=lambda x: len(x[1]),
                             reverse=True)[:top_n]

    for cluster_id, queries in sorted_clusters:
        # Get the shortest query as template (likely cleanest)
        template_query = min(queries, key=lambda q: len(q['original_sql']))

        # Generate natural language description
        tables = Counter()
        for q in queries:
            tables.update(q['tables'])
        main_tables = [t[0] for t in tables.most_common(2)]

        has_join = template_query['joins'] > 0
        has_agg = len(template_query['aggregations']) > 0
        has_group = template_query['group_by']

        # Create NL description
        if has_agg and has_group:
            if has_join:
                nl = f"Show aggregated metrics from {' and '.join(main_tables)}"
            else:
                nl = f"Calculate summary statistics for {main_tables[0]}"
        elif has_join:
            nl = f"Combine data from {' and '.join(main_tables)}"
        else:
            nl = f"Query {main_tables[0]} table"

        # Parameterize the query (replace literals)
        sql = template_query['original_sql']
        # Replace date literals
        sql = re.sub(r"'2026-\d{2}-\d{2}'", "CURRENT_DATE - INTERVAL {{days}} DAYS", sql,
count=1)
        # Replace numeric literals in WHERE
        sql = re.sub(r">\s*\d+", "> {{threshold}}", sql, count=1)

        examples.append({
            'natural_language': nl,
            'sql': sql,
            'frequency': len(queries),
            'avg_execution_time_ms': sum(q['duration_ms'] for q in queries) / len(querie
s),
            'tables_used': main_tables,
            'cluster_id': cluster_id
        })
```

```
    return examples

# Generate examples
print("="*60)
print("GENERATING EXAMPLE QUERIES")
print("="*60)

example_queries = generate_example_queries(clustered_queries, parsed_queries, top_n=10)

print(f"✅ Generated {len(example_queries)} example queries\n")

# Display examples
for i, ex in enumerate(example_queries, 1):
    print(f"\n{'='*70}")
    print(f"EXAMPLE #{i}")
    print(f"{'='*70}")
    print(f"📝 Natural Language: {ex['natural_language']}")
    print(f"📊 Frequency: {ex['frequency']} queries")
    print(f"⏱️  Avg Duration: {ex['avg_execution_time_ms']:.0f}ms")
    print(f"📁 Tables: {', '.join(ex['tables_used'])}")
    print(f"\n💻 SQL Template:")
    print(ex['sql'][:200] + "..." if len(ex['sql']) > 200 else ex['sql'])
```

Expected Output:

```
============================================================
GENERATING EXAMPLE QUERIES
============================================================
✅ Generated 10 example queries

======================================================================
EXAMPLE #1
======================================================================
📝 Natural Language: Show aggregated metrics from orders and customers
📊 Frequency: 234 queries
⏱️  Avg Duration: 3420ms
📁 Tables: orders, customers

💻 SQL Template:
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) as order_count,
    SUM(o.total_amount) as lifetime_value
FROM main.sales.customers c
JOIN main.sales.orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= CURRENT_DATE - INTERVAL {{days}} DAYS
GROUP BY c.customer_id, c.name
ORDER BY lifetime_value DESC...

======================================================================
EXAMPLE #2
======================================================================
📝 Natural Language: Show aggregated metrics from order_items and orders
📊 Frequency: 189 queries
⏱️  Avg Duration: 4850ms
📁 Tables: order_items, orders
```

```
💻 SQL Template:
SELECT
    o.order_date,
    COUNT(DISTINCT oi.product_id) as unique_products,
    SUM(oi.quantity) as total_units,
    SUM(oi.quantity * oi.unit_price) as total_revenue
FROM main.sales.orders o
JOIN main.sales.order_items oi ON o.order_id = oi.order_id
WHERE o.order_date >= CURRENT_DATE - INTERVAL {{days}} DAYS
GROUP BY o.order_date...

======================================================================
EXAMPLE #3
======================================================================
📝 Natural Language: Show aggregated metrics from order_items and products
📊 Frequency: 156 queries
⏱️  Avg Duration: 5230ms
📁 Tables: order_items, products

💻 SQL Template:
SELECT
    p.product_name,
    p.category,
    SUM(oi.quantity) as units_sold,
    SUM(oi.quantity * oi.unit_price) as revenue
FROM main.sales.products p
JOIN main.sales.order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.category
ORDER BY revenue DESC...
```

Notebook 5: Instruction Generation

(Continuing due to length limit - would you like me to continue with Notebooks 5-7?)

Let me know if you'd like me to:

1. **Continue with remaining notebooks** (5-7: Instruction Generation, Assembly & Validation, Deployment)
2. **Add more detail to existing notebooks** (error handling, visualizations)
3. **Create a master execution notebook** that runs all stages sequentially

---

Notebook 5: Instruction Generation

Cell 5.1: Generate Table Instructions

```
def generate_table_instructions(table_metadata, column_profiles, relationships, example_qu
eries):
    """Generate human-readable instructions for each table"""

    instructions = {}

    for table_name, metadata in table_metadata.items():
        simple_name = table_name.split('.')[-1]

        # Build table description
        row_count = metadata['row_count']
        last_modified = metadata['last_modified'].strftime('%Y-%m-%d')
```

```
        description_parts = []

        # Main description
        description_parts.append(
            f"The {simple_name} table contains {row_count:,} records as of {last_modifie
d}."
        )

        # Add business context based on table name
        if 'order' in simple_name.lower():
            description_parts.append(
                "This table tracks transaction records including dates, amounts, and custo
mer associations."
            )
        elif 'customer' in simple_name.lower():
            description_parts.append(
                "This table stores customer information including contact details and acco
unt status."
            )
        elif 'product' in simple_name.lower():
            description_parts.append(
                "This table contains product catalog information including names, categori
es, and pricing."
            )

        # Document key columns
        key_columns = {}
        profiles = column_profiles.get(table_name, [])

        for col in metadata['columns'][:10]:  # Top 10 columns
            col_name = col['column_name']
            col_type = col['data_type']
            col_comment = col.get('comment', '')

            # Find profile data
            col_profile = next((p for p in profiles if p['column'] == col_name), None)

            description = col_comment if col_comment else f"{col_name} column"

            # Add cardinality info
            if col_profile:
                if col_profile['recommend_entity_matching']:
                    description += f" (Low cardinality: {col_profile['distinct_count']} un
ique values - good for filtering)"
                if col_profile['pii_detected']:
                    description += f" ⚠ Contains {col_profile['pii_detected'].upper()}"

            key_columns[col_name] = description

        # Find relationships involving this table
        table_rels = [r for r in relationships if table_name in [r['source_table'], r['tar
get_table']]]

        join_hints = []
        for rel in table_rels[:5]:  # Top 5 relationships
            if rel['source_table'] == table_name:
                target_simple = rel['target_table'].split('.')[-1]
```

```
                join_hints.append(
                    f"Join to {target_simple} using {rel['source_column']}"
                )
            else:
                source_simple = rel['source_table'].split('.')[-1]
                join_hints.append(
                    f"Join from {source_simple} using {rel['target_column']}"
                )

        # Find common use cases from example queries
        related_examples = [ex for ex in example_queries if simple_name in ex.get('tables_
used', [])]
        use_cases = []
        for ex in related_examples[:3]:
            use_cases.append(ex['natural_language'])

        instructions[table_name] = {
            'description': ' '.join(description_parts),
            'key_columns': key_columns,
            'join_hints': join_hints,
            'common_use_cases': use_cases,
            'row_count': row_count,
            'last_updated': last_modified
        }

    return instructions

# Generate instructions
print("="*60)
print("GENERATING TABLE INSTRUCTIONS")
print("="*60)

table_instructions = generate_table_instructions(
    table_metadata,
    column_profiles,
    final_relationships,
    example_queries
)

print(f"✅ Generated instructions for {len(table_instructions)} tables\n")

# Display sample instruction
sample_table = f"{config['catalog']}.{config['schema']}.orders"
if sample_table in table_instructions:
    instr = table_instructions[sample_table]

    print("="*70)
    print("SAMPLE INSTRUCTION: orders table")
    print("="*70)
    print(f"\n📖 DESCRIPTION:")
    print(f"   {instr['description']}")

    print(f"\n🔑 KEY COLUMNS ({len(instr['key_columns'])} total):")
    for col, desc in list(instr['key_columns'].items())[:5]:
        print(f"   • {col}: {desc}")

    print(f"\n🔗 JOIN HINTS:")
    for hint in instr['join_hints']:
```

```
        print(f"   • {hint}")

    print(f"\n💼 COMMON USE CASES:")
    for use_case in instr['common_use_cases']:
        print(f"   • {use_case}")
```

Expected Output:

```
============================================================
GENERATING TABLE INSTRUCTIONS
============================================================
✅ Generated instructions for 5 tables

======================================================================
SAMPLE INSTRUCTION: orders table
======================================================================

📖 DESCRIPTION:
   The orders table contains 1,250,000 records as of 2026-02-15. This table tracks transac
tion records including dates, amounts, and customer associations.

🔑 KEY COLUMNS (8 total):
   • order_id: Unique order identifier
   • customer_id: Foreign key to customers table
   • order_date: Date order was placed (Low cardinality: 1095 unique values - good for fil
tering)
   • order_status: Order status (Low cardinality: 5 unique values - good for filtering)
   • total_amount: Total order value in USD

🔗 JOIN HINTS:
   • Join to customers using customer_id
   • Join to warehouses using warehouse_id
   • Join from order_items using order_id

💼 COMMON USE CASES:
   • Show aggregated metrics from orders and customers
   • Calculate summary statistics for orders
   • Combine data from orders and warehouses
```

Cell 5.2: Generate Join Instructions

```
def generate_join_instructions(relationships):
    """Convert relationships to Genie-compatible join instructions"""

    join_instructions = []

    for rel in relationships:
        source_table = rel['source_table']
        target_table = rel['target_table']
        source_col = rel['source_column']
        target_col = rel['target_column']

        source_simple = source_table.split('.')[-1]
        target_simple = target_table.split('.')[-1]

        # Determine join type
        # If source column can be null, suggest LEFT JOIN, otherwise INNER
```

```
        join_type = "INNER"

        # Build explanation
        explanations = []

        if 'order' in source_simple and 'customer' in target_simple:
            explanations.append(
                f"Join {source_simple} with {target_simple} to get customer demographics a
nd contact information."
            )
            explanations.append(
                "Use INNER JOIN to exclude orders with invalid customer references."
            )
        elif 'order_item' in source_simple and 'order' in target_simple:
            explanations.append(
                f"Join {source_simple} with {target_simple} to get order-level details lik
e dates and customer."
            )
            explanations.append(
                "Always use INNER JOIN since every order item must have a parent order."
            )
        elif 'order_item' in source_simple and 'product' in target_simple:
            explanations.append(
                f"Join {source_simple} with {target_simple} to get product names, categori
es, and descriptions."
            )
            explanations.append(
                "Use INNER JOIN for product analysis, LEFT JOIN if you need to detect orph
aned items."
            )
        else:
            explanations.append(
                f"Join {source_simple} with {target_simple} using the {source_col} relatio
nship."
            )

        join_instructions.append({
            'source_table': source_table,
            'source_column': source_col,
            'target_table': target_table,
            'target_column': target_col,
            'join_definition': f"{source_table}.{source_col} = {target_table}.{target_co
l}",
            'recommended_join_type': join_type,
            'explanation': ' '.join(explanations),
            'confidence': rel['confidence'],
            'methods': rel.get('methods', [])
        })

    return join_instructions

# Generate join instructions
print("="*60)
print("GENERATING JOIN INSTRUCTIONS")
print("="*60)

join_instructions = generate_join_instructions(final_relationships)
```

```
print(f"✅ Generated {len(join_instructions)} join instructions\n")

# Display all join instructions
for i, join_instr in enumerate(join_instructions, 1):
    print(f"\n{'='*70}")
    print(f"JOIN INSTRUCTION #{i}")
    print(f"{'='*70}")
    print(f"📋 Definition: {join_instr['join_definition']}")
    print(f"🔗 Type: {join_instr['recommended_join_type']} JOIN")
    print(f"📊 Confidence: {join_instr['confidence']:.2f}")
    print(f"🔍 Discovery Methods: {', '.join(join_instr['methods'])}")
    print(f"\n💡 Explanation:")
    print(f"   {join_instr['explanation']}")
```

Expected Output:

```
============================================================
GENERATING JOIN INSTRUCTIONS
============================================================
✅ Generated 5 join instructions

======================================================================
JOIN INSTRUCTION #1
======================================================================
📋 Definition: main.sales.orders.customer_id = main.sales.customers.customer_id
🔗 Type: INNER JOIN
📊 Confidence: 1.00
🔍 Discovery Methods: declared_fk, naming_pattern, query_pattern

💡 Explanation:
   Join orders with customers to get customer demographics and contact information. Use IN
NER JOIN to exclude orders with invalid customer references.

======================================================================
JOIN INSTRUCTION #2
======================================================================
📋 Definition: main.sales.order_items.order_id = main.sales.orders.order_id
🔗 Type: INNER JOIN
📊 Confidence: 1.00
🔍 Discovery Methods: declared_fk, naming_pattern, query_pattern

💡 Explanation:
   Join order_items with orders to get order-level details like dates and customer. Always
use INNER JOIN since every order item must have a parent order.

======================================================================
JOIN INSTRUCTION #3
======================================================================
📋 Definition: main.sales.order_items.product_id = main.sales.products.product_id
🔗 Type: INNER JOIN
📊 Confidence: 1.00
🔍 Discovery Methods: declared_fk, naming_pattern, query_pattern

💡 Explanation:
   Join order_items with products to get product names, categories, and descriptions. Use
INNER JOIN for product analysis, LEFT JOIN if you need to detect orphaned items.

======================================================================
```

```
JOIN INSTRUCTION #4
======================================================================
📋 Definition: main.sales.orders.warehouse_id = main.sales.warehouses.warehouse_id
🔗 Type: INNER JOIN
📊 Confidence: 0.95
🔍 Discovery Methods: declared_fk, naming_pattern, query_pattern

💡 Explanation:
   Join orders with warehouses using the warehouse_id relationship.

======================================================================
JOIN INSTRUCTION #5
======================================================================
📋 Definition: main.sales.customers.customer_id = main.sales.orders.customer_id
🔗 Type: INNER JOIN
📊 Confidence: 0.85
🔍 Discovery Methods: naming_pattern, query_pattern

💡 Explanation:
   Join customers with orders using the customer_id relationship.
```

Cell 5.3: Generate SQL Expressions

```
def generate_sql_expressions(table_metadata, column_profiles, query_history):
    """Auto-generate common measures, filters, and dimensions"""

    expressions = {
        'measures': [],
        'filters': [],
        'dimensions': []
    }

    for table_name, metadata in table_metadata.items():
        simple_name = table_name.split('.')[-1]

        for col in metadata['columns']:
            col_name = col['column_name']
            col_type = col['data_type']

            # Generate MEASURES for numeric columns
            if col_type in ['BIGINT', 'INT', 'DECIMAL', 'DOUBLE', 'FLOAT']:
                # SUM measure
                if 'amount' in col_name.lower() or 'price' in col_name.lower() or 'revenu
e' in col_name.lower():
                    expressions['measures'].append({
                        'name': f"Total {col_name.replace('_', ' ').title()}",
                        'expression': f"SUM({table_name}.{col_name})",
                        'description': f"Sum of all {col_name.replace('_', ' ')} values",
                        'table': simple_name
                    })

                # AVG measure
                if 'amount' in col_name.lower() or 'value' in col_name.lower():
                    expressions['measures'].append({
                        'name': f"Average {col_name.replace('_', ' ').title()}",
                        'expression': f"AVG({table_name}.{col_name})",
                        'description': f"Average {col_name.replace('_', ' ')} per record",
```

```
                        'table': simple_name
                    })

                # COUNT measure for ID columns
                if col_name.endswith('_id') or col_name == 'id':
                    entity = col_name.replace('_id', '').replace('_', ' ').title()
                    expressions['measures'].append({
                        'name': f"{entity} Count",
                        'expression': f"COUNT(DISTINCT {table_name}.{col_name})",
                        'description': f"Number of unique {entity.lower()}s",
                        'table': simple_name
                    })

            # Generate FILTERS
            if col_type == 'DATE' or col_type == 'TIMESTAMP':
                # Recent records filter
                expressions['filters'].append({
                    'name': f"Recent {simple_name.title()} (Last 30 Days)",
                    'expression': f"{table_name}.{col_name} >= CURRENT_DATE - INTERVAL 30
DAYS",
                    'description': f"Filter {simple_name} from the last 30 days",
                    'table': simple_name
                })

                # Year-to-date filter
                expressions['filters'].append({
                    'name': f"Year to Date {simple_name.title()}",
                    'expression': f"YEAR({table_name}.{col_name}) = YEAR(CURRENT_DATE)",
                    'description': f"Filter {simple_name} from current year",
                    'table': simple_name
                })

            # High-value filter for amount columns
            if 'amount' in col_name.lower() and col_type in ['DECIMAL', 'DOUBLE']:
                expressions['filters'].append({
                    'name': f"High Value {simple_name.title()}",
                    'expression': f"{table_name}.{col_name} > 1000",
                    'description': f"Filter {simple_name} with {col_name} exceeding $1,00
0",
                    'table': simple_name
                })

            # Status filters for low-cardinality string columns
            profiles = column_profiles.get(table_name, [])
            col_profile = next((p for p in profiles if p['column'] == col_name), None)

            if col_profile and col_profile['cardinality_category'] == 'very_low' and col_t
ype == 'STRING':
                expressions['filters'].append({
                    'name': f"{col_name.replace('_', ' ').title()} Filter",
                    'expression': f"{table_name}.{col_name} = '{{value}}'",
                    'description': f"Filter by specific {col_name.replace('_', ' ')}",
                    'table': simple_name
                })

            # Generate DIMENSIONS for date columns
            if col_type == 'DATE' or col_type == 'TIMESTAMP':
                expressions['dimensions'].append({
```

```
                    'name': f"{col_name.replace('_', ' ').title()} - Year",
                    'expression': f"YEAR({table_name}.{col_name})",
                    'description': f"Year from {col_name}",
                    'table': simple_name
                })

                expressions['dimensions'].append({
                    'name': f"{col_name.replace('_', ' ').title()} - Month",
                    'expression': f"DATE_TRUNC('MONTH', {table_name}.{col_name})",
                    'description': f"Month from {col_name}",
                    'table': simple_name
                })

                expressions['dimensions'].append({
                    'name': f"{col_name.replace('_', ' ').title()} - Quarter",
                    'expression': f"CONCAT('Q', QUARTER({table_name}.{col_name}), ' ', YEA
R({table_name}.{col_name}))",
                    'description': f"Quarter from {col_name}",
                    'table': simple_name
                })

    return expressions

# Generate SQL expressions
print("="*60)
print("GENERATING SQL EXPRESSIONS")
print("="*60)

sql_expressions = generate_sql_expressions(
    table_metadata,
    column_profiles,
    query_history
)

print(f"✅ Generated SQL expressions:")
print(f"   • {len(sql_expressions['measures'])} measures")
print(f"   • {len(sql_expressions['filters'])} filters")
print(f"   • {len(sql_expressions['dimensions'])} dimensions")
print()

# Display samples
print("="*70)
print("SAMPLE MEASURES (Top 5)")
print("="*70)
for measure in sql_expressions['measures'][:5]:
    print(f"\n📊 {measure['name']}")
    print(f"   Expression: {measure['expression']}")
    print(f"   Description: {measure['description']}")
    print(f"   Table: {measure['table']}")

print("\n" + "="*70)
print("SAMPLE FILTERS (Top 5)")
print("="*70)
for filter_expr in sql_expressions['filters'][:5]:
    print(f"\n🔍 {filter_expr['name']}")
    print(f"   Expression: {filter_expr['expression']}")
    print(f"   Description: {filter_expr['description']}")
    print(f"   Table: {filter_expr['table']}")
```

```
print("\n" + "="*70)
print("SAMPLE DIMENSIONS (Top 5)")
print("="*70)
for dim in sql_expressions['dimensions'][:5]:
    print(f"\n📅 {dim['name']}")
    print(f"   Expression: {dim['expression']}")
    print(f"   Description: {dim['description']}")
    print(f"   Table: {dim['table']}")
```

Expected Output:

```
============================================================
GENERATING SQL EXPRESSIONS
============================================================
✅ Generated SQL expressions:
   • 18 measures
   • 24 filters
   • 15 dimensions

======================================================================
SAMPLE MEASURES (Top 5)
======================================================================

📊 Total Total Amount
   Expression: SUM(main.sales.orders.total_amount)
   Description: Sum of all total amount values
   Table: orders

📊 Average Total Amount
   Expression: AVG(main.sales.orders.total_amount)
   Description: Average total amount per record
   Table: orders

📊 Order Count
   Expression: COUNT(DISTINCT main.sales.orders.order_id)
   Description: Number of unique orders
   Table: orders

📊 Customer Count
   Expression: COUNT(DISTINCT main.sales.orders.customer_id)
   Description: Number of unique customers
   Table: orders

📊 Total Unit Price
   Expression: SUM(main.sales.order_items.unit_price)
   Description: Sum of all unit price values
   Table: order_items

======================================================================
SAMPLE FILTERS (Top 5)
======================================================================

🔍 Recent Orders (Last 30 Days)
   Expression: main.sales.orders.order_date >= CURRENT_DATE - INTERVAL 30 DAYS
   Description: Filter orders from the last 30 days
   Table: orders
```

```
🔍 Year to Date Orders
   Expression: YEAR(main.sales.orders.order_date) = YEAR(CURRENT_DATE)
   Description: Filter orders from current year
   Table: orders

🔍 High Value Orders
   Expression: main.sales.orders.total_amount > 1000
   Description: Filter orders with total_amount exceeding $1,000
   Table: orders

🔍 Order Status Filter
   Expression: main.sales.orders.order_status = '{value}'
   Description: Filter by specific order status
   Table: orders

🔍 Payment Method Filter
   Expression: main.sales.orders.payment_method = '{value}'
   Description: Filter by specific payment method
   Table: orders

======================================================================
SAMPLE DIMENSIONS (Top 5)
======================================================================

📅 Order Date - Year
   Expression: YEAR(main.sales.orders.order_date)
   Description: Year from order_date
   Table: orders

📅 Order Date - Month
   Expression: DATE_TRUNC('MONTH', main.sales.orders.order_date)
   Description: Month from order_date
   Table: orders

📅 Order Date - Quarter
   Expression: CONCAT('Q', QUARTER(main.sales.orders.order_date), ' ', YEAR(main.sales.ord
ers.order_date))
   Description: Quarter from order_date
   Table: orders

📅 Created At - Year
   Expression: YEAR(main.sales.orders.created_at)
   Description: Year from created_at
   Table: orders

📅 Created At - Month
   Expression: DATE_TRUNC('MONTH', main.sales.orders.created_at)
   Description: Month from created_at
   Table: orders
```

Notebook 6: Knowledge Store Assembly & Validation

Cell 6.1: Assemble Knowledge Store

```
def assemble_knowledge_store(
    config,
    table_metadata,
```

```
    table_instructions,
    join_instructions,
    sql_expressions,
    example_queries
):
    """Assemble all components into Genie-compatible knowledge store"""

    knowledge_store = {
        'space_name': f"{config['schema'].title()} Analytics Space",
        'space_description': f"Auto-generated Genie space for {config['catalog']}.{config
['schema']} schema",
        'catalog': config['catalog'],
        'schema': config['schema'],
        'warehouse_id': config['genie_warehouse_id'],
        'created_at': datetime.now().isoformat(),
        'configuration': {
            'lookback_days': config['lookback_days'],
            'confidence_threshold': config['confidence_threshold'],
            'tables_included': config['tables']
        },
        'tables': [],
        'joins': [],
        'sql_expressions': {
            'measures': [],
            'filters': [],
            'dimensions': []
        },
        'example_queries': [],
        'global_instructions': ""
    }

    # Add table information
    for table_name, instruction in table_instructions.items():
        simple_name = table_name.split('.')[-1]
        metadata = table_metadata[table_name]

        table_entry = {
            'catalog': config['catalog'],
            'schema': config['schema'],
            'table_name': simple_name,
            'full_name': table_name,
            'description': instruction['description'],
            'row_count': instruction['row_count'],
            'last_updated': instruction['last_updated'],
            'columns': [],
            'key_columns': instruction['key_columns'],
            'join_hints': instruction['join_hints'],
            'common_use_cases': instruction['common_use_cases']
        }

        # Add column details
        for col in metadata['columns']:
            table_entry['columns'].append({
                'name': col['column_name'],
                'type': col['data_type'],
                'nullable': col['is_nullable'],
                'comment': col.get('comment', '')
            })
```

```
        knowledge_store['tables'].append(table_entry)

    # Add join definitions
    for join_instr in join_instructions:
        knowledge_store['joins'].append({
            'source_table': join_instr['source_table'],
            'source_column': join_instr['source_column'],
            'target_table': join_instr['target_table'],
            'target_column': join_instr['target_column'],
            'join_type': join_instr['recommended_join_type'],
            'explanation': join_instr['explanation'],
            'confidence': join_instr['confidence']
        })

    # Add SQL expressions
    knowledge_store['sql_expressions']['measures'] = sql_expressions['measures']
    knowledge_store['sql_expressions']['filters'] = sql_expressions['filters']
    knowledge_store['sql_expressions']['dimensions'] = sql_expressions['dimensions']

    # Add example queries
    for example in example_queries:
        knowledge_store['example_queries'].append({
            'natural_language': example['natural_language'],
            'sql': example['sql'],
            'frequency': example['frequency'],
            'avg_execution_time_ms': example['avg_execution_time_ms'],
            'tables_used': example['tables_used']
        })

    # Generate global instructions
    global_instructions = f"""
This Genie space provides natural language access to the {config['schema']} dataset.

AVAILABLE TABLES:
{chr(10).join([f"• {t['table_name']}: {t['description'][:100]}..." for t in knowledge_stor
e['tables']])}

COMMON QUERIES:
{chr(10).join([f"• {eq['natural_language']}" for eq in example_queries[:5]])}

TIPS FOR BEST RESULTS:
• Be specific about date ranges (e.g., "last 30 days", "Q4 2025")
• Specify what metrics you want (e.g., "total revenue", "count of orders")
• Use table names when ambiguous (e.g., "customer name from customers table")
• Filter data to improve performance (avoid querying all historical data)

This space was automatically generated on {datetime.now().strftime('%Y-%m-%d %H:%M')} usin
g Auto-Genie MVP.
"""

    knowledge_store['global_instructions'] = global_instructions

    return knowledge_store

# Assemble knowledge store
print("="*60)
print("ASSEMBLING KNOWLEDGE STORE")
```

```
print("="*60)

knowledge_store = assemble_knowledge_store(
    config,
    table_metadata,
    table_instructions,
    join_instructions,
    sql_expressions,
    example_queries
)

print(f"✅ Knowledge store assembled successfully\n")

# Display summary
print("📊 KNOWLEDGE STORE SUMMARY:")
print(f"   Space Name: {knowledge_store['space_name']}")
print(f"   Catalog.Schema: {knowledge_store['catalog']}.{knowledge_store['schema']}")
print(f"   Tables: {len(knowledge_store['tables'])}")
print(f"   Joins: {len(knowledge_store['joins'])}")
print(f"   Measures: {len(knowledge_store['sql_expressions']['measures'])}")
print(f"   Filters: {len(knowledge_store['sql_expressions']['filters'])}")
print(f"   Dimensions: {len(knowledge_store['sql_expressions']['dimensions'])}")
print(f"   Example Queries: {len(knowledge_store['example_queries'])}")
print(f"   Created: {knowledge_store['created_at']}")

# Save to JSON
output_file = f"{config['output_path']}/knowledge_store.json"
dbutils.fs.mkdirs(config['output_path'])

with open(output_file.replace('/dbfs', ''), 'w') as f:
    json.dump(knowledge_store, f, indent=2)

print(f"\n💾 Saved to: {output_file}")
```

Expected Output:

```
============================================================
ASSEMBLING KNOWLEDGE STORE
============================================================
✅ Knowledge store assembled successfully

📊 KNOWLEDGE STORE SUMMARY:
   Space Name: Sales Analytics Space
   Catalog.Schema: main.sales
   Tables: 5
   Joins: 5
   Measures: 18
   Filters: 24
   Dimensions: 15
   Example Queries: 10
   Created: 2026-02-21T16:45:30.123456

💾 Saved to: /dbfs/tmp/auto_genie_outputs/knowledge_store.json
```

Cell 6.2: Validate Knowledge Store Structure

```
def validate_knowledge_store_structure(knowledge_store):
    """Validate the structure and completeness of knowledge store"""

    validation_results = {
        'status': 'passed',
        'checks': [],
        'warnings': [],
        'errors': []
    }

    # Check 1: Required fields
    required_fields = ['space_name', 'catalog', 'schema', 'tables', 'joins']
    for field in required_fields:
        if field not in knowledge_store or not knowledge_store[field]:
            validation_results['errors'].append(f"Missing required field: {field}")
            validation_results['status'] = 'failed'
        else:
            validation_results['checks'].append({
                'check': f'required_field_{field}',
                'status': 'passed',
                'details': f'Field {field} is present'
            })

    # Check 2: Table count
    if len(knowledge_store['tables']) == 0:
        validation_results['errors'].append("No tables defined in knowledge store")
        validation_results['status'] = 'failed'
    elif len(knowledge_store['tables']) < 3:
        validation_results['warnings'].append(f"Only {len(knowledge_store['tables'])} tabl
es defined - may limit usefulness")

    validation_results['checks'].append({
        'check': 'table_count',
        'status': 'passed',
        'details': f'{len(knowledge_store["tables"])} tables defined'
    })

    # Check 3: Table descriptions
    tables_without_desc = [t['table_name'] for t in knowledge_store['tables'] if not t.get
('description')]
    if tables_without_desc:
        validation_results['warnings'].append(f"Tables without descriptions: {', '.join(ta
bles_without_desc)}")

    validation_results['checks'].append({
        'check': 'table_descriptions',
        'status': 'passed' if not tables_without_desc else 'warning',
        'details': f'{len(knowledge_store["tables"]) - len(tables_without_desc)}/{len(know
ledge_store["tables"])} tables have descriptions'
    })

    # Check 4: Join definitions
    if len(knowledge_store['joins']) == 0:
        validation_results['warnings'].append("No joins defined - users cannot query acros
s tables")

    validation_results['checks'].append({
```

```
        'check': 'join_definitions',
        'status': 'passed',
        'details': f'{len(knowledge_store["joins"])} join relationships defined'
    })

    # Check 5: SQL expressions
    total_expressions = (
        len(knowledge_store['sql_expressions']['measures']) +
        len(knowledge_store['sql_expressions']['filters']) +
        len(knowledge_store['sql_expressions']['dimensions'])
    )

    if total_expressions == 0:
        validation_results['warnings'].append("No SQL expressions defined - limited analyt
ical capabilities")

    validation_results['checks'].append({
        'check': 'sql_expressions',
        'status': 'passed',
        'details': f'{total_expressions} SQL expressions defined'
    })

    # Check 6: Example queries
    if len(knowledge_store.get('example_queries', [])) == 0:
        validation_results['warnings'].append("No example queries - Genie may have lower a
ccuracy")
    elif len(knowledge_store['example_queries']) < 5:
        validation_results['warnings'].append(f"Only {len(knowledge_store['example_querie
s'])} example queries - recommend 10+")

    validation_results['checks'].append({
        'check': 'example_queries',
        'status': 'passed',
        'details': f'{len(knowledge_store.get("example_queries", []))} example queries def
ined'
    })

    # Check 7: Global instructions
    if not knowledge_store.get('global_instructions'):
        validation_results['warnings'].append("No global instructions provided")
    elif len(knowledge_store['global_instructions']) < 100:
        validation_results['warnings'].append("Global instructions are very brief")

    validation_results['checks'].append({
        'check': 'global_instructions',
        'status': 'passed',
        'details': f'Global instructions: {len(knowledge_store.get("global_instructions",
""))} characters'
    })

    return validation_results

# Validate structure
print("="*60)
print("VALIDATING KNOWLEDGE STORE STRUCTURE")
print("="*60)

validation_results = validate_knowledge_store_structure(knowledge_store)
```

```
print(f"\n{'='*70}")
print(f"VALIDATION RESULTS: {validation_results['status'].upper()}")
print(f"{'='*70}\n")

print(f"✅ PASSED CHECKS ({len([c for c in validation_results['checks'] if c['status'] ==
'passed'])}):")
for check in validation_results['checks']:
    if check['status'] == 'passed':
        print(f"   • {check['check']}: {check['details']}")

if validation_results['warnings']:
    print(f"\n⚠  WARNINGS ({len(validation_results['warnings'])}):")
    for warning in validation_results['warnings']:
        print(f"   • {warning}")

if validation_results['errors']:
    print(f"\n❌ ERRORS ({len(validation_results['errors'])}):")
    for error in validation_results['errors']:
        print(f"   • {error}")

print(f"\n{'='*70}")
print(f"Overall Status: {'✅ READY FOR DEPLOYMENT' if validation_results['status'] == 'pas
sed' else '❌ REQUIRES FIXES'}")
print(f"{'='*70}")
```

Expected Output:

```
============================================================
VALIDATING KNOWLEDGE STORE STRUCTURE
============================================================

======================================================================
VALIDATION RESULTS: PASSED
======================================================================

✅ PASSED CHECKS (7):
   • required_field_space_name: Field space_name is present
   • required_field_catalog: Field catalog is present
   • required_field_schema: Field schema is present
   • required_field_tables: Field tables is present
   • required_field_joins: Field joins is present
   • table_count: 5 tables defined
   • table_descriptions: 5/5 tables have descriptions
   • join_definitions: 5 join relationships defined
   • sql_expressions: 57 SQL expressions defined
   • example_queries: 10 example queries defined
   • global_instructions: Global instructions: 847 characters

======================================================================
Overall Status: ✅ READY FOR DEPLOYMENT
======================================================================
```

Cell 6.3: Validate SQL Syntax

```
def validate_sql_syntax(knowledge_store):
    """Validate SQL syntax for joins, expressions, and example queries"""
```

```
    validation_results = {
        'joins': {'passed': 0, 'failed': 0, 'errors': []},
        'measures': {'passed': 0, 'failed': 0, 'errors': []},
        'filters': {'passed': 0, 'failed': 0, 'errors': []},
        'dimensions': {'passed': 0, 'failed': 0, 'errors': []},
        'example_queries': {'passed': 0, 'failed': 0, 'errors': []}
    }

    print("Validating SQL syntax...\n")

    # Validate joins
    print("Checking joins...", end=" ")
    for join in knowledge_store['joins']:
        try:
            # Build a simple JOIN query to test syntax
            test_query = f"""
            SELECT * FROM {join['source_table']}
            {join['join_type']} JOIN {join['target_table']}
            ON {join['source_table']}.{join['source_column']} = {join['target_table']}.{jo
in['target_column']}
            LIMIT 1
            """

            # Parse with sqlglot
            parsed = sqlglot.parse_one(test_query, read='databricks')
            validation_results['joins']['passed'] += 1

        except Exception as e:
            validation_results['joins']['failed'] += 1
            validation_results['joins']['errors'].append({
                'join': f"{join['source_table']}.{join['source_column']} = {join['target_t
able']}.{join['target_column']}",
                'error': str(e)
            })
    print(f"✅ {validation_results['joins']['passed']}/{validation_results['joins']['passe
d'] + validation_results['joins']['failed']}")

    # Validate measures
    print("Checking measures...", end=" ")
    for measure in knowledge_store['sql_expressions']['measures']:
        try:
            # Test expression syntax
            test_query = f"SELECT {measure['expression']} as result FROM {knowledge_store
['catalog']}.{knowledge_store['schema']}.{measure['table']} LIMIT 1"
            parsed = sqlglot.parse_one(test_query, read='databricks')
            validation_results['measures']['passed'] += 1
        except Exception as e:
            validation_results['measures']['failed'] += 1
            validation_results['measures']['errors'].append({
                'measure': measure['name'],
                'expression': measure['expression'],
                'error': str(e)
            })
    print(f"✅ {validation_results['measures']['passed']}/{validation_results['measures']
['passed'] + validation_results['measures']['failed']}")

    # Validate filters
```

```
    print("Checking filters...", end=" ")
    for filter_expr in knowledge_store['sql_expressions']['filters']:
        try:
            # Test filter syntax
            test_query = f"SELECT * FROM {knowledge_store['catalog']}.{knowledge_store['sc
hema']}.{filter_expr['table']} WHERE {filter_expr['expression'].replace('{value}', 'test_v
alue')} LIMIT 1"
            parsed = sqlglot.parse_one(test_query, read='databricks')
            validation_results['filters']['passed'] += 1
        except Exception as e:
            validation_results['filters']['failed'] += 1
            validation_results['filters']['errors'].append({
                'filter': filter_expr['name'],
                'expression': filter_expr['expression'],
                'error': str(e)
            })
    print(f"✅ {validation_results['filters']['passed']}/{validation_results['filters']['p
assed'] + validation_results['filters']['failed']}")

    # Validate dimensions
    print("Checking dimensions...", end=" ")
    for dim in knowledge_store['sql_expressions']['dimensions']:
        try:
            # Test dimension syntax
            test_query = f"SELECT {dim['expression']} as result FROM {knowledge_store['cat
alog']}.{knowledge_store['schema']}.{dim['table']} LIMIT 1"
            parsed = sqlglot.parse_one(test_query, read='databricks')
            validation_results['dimensions']['passed'] += 1
        except Exception as e:
            validation_results['dimensions']['failed'] += 1
            validation_results['dimensions']['errors'].append({
                'dimension': dim['name'],
                'expression': dim['expression'],
                'error': str(e)
            })
    print(f"✅ {validation_results['dimensions']['passed']}/{validation_results['dimension
s']['passed'] + validation_results['dimensions']['failed']}")

    # Validate example queries
    print("Checking example queries...", end=" ")
    for example in knowledge_store['example_queries']:
        try:
            parsed = sqlglot.parse_one(example['sql'], read='databricks')
            validation_results['example_queries']['passed'] += 1
        except Exception as e:
            validation_results['example_queries']['failed'] += 1
            validation_results['example_queries']['errors'].append({
                'query': example['natural_language'],
                'sql': example['sql'][:100],
                'error': str(e)
            })
    print(f"✅ {validation_results['example_queries']['passed']}/{validation_results['exam
ple_queries']['passed'] + validation_results['example_queries']['failed']}")

    return validation_results

# Validate SQL syntax
print("="*60)
```

```
print("VALIDATING SQL SYNTAX")
print("="*60)
print()

sql_validation = validate_sql_syntax(knowledge_store)

print(f"\n{'='*70}")
print("SQL VALIDATION SUMMARY")
print(f"{'='*70}")

total_passed = sum(v['passed'] for v in sql_validation.values())
total_failed = sum(v['failed'] for v in sql_validation.values())

print(f"\n📊 Overall: {total_passed} passed, {total_failed} failed")
print(f"   • Joins: {sql_validation['joins']['passed']}/{sql_validation['joins']['passed']
+ sql_validation['joins']['failed']}")
print(f"   • Measures: {sql_validation['measures']['passed']}/{sql_validation['measures']
['passed'] + sql_validation['measures']['failed']}")
print(f"   • Filters: {sql_validation['filters']['passed']}/{sql_validation['filters']['pa
ssed'] + sql_validation['filters']['failed']}")
print(f"   • Dimensions: {sql_validation['dimensions']['passed']}/{sql_validation['dimensi
ons']['passed'] + sql_validation['dimensions']['failed']}")
print(f"   • Example Queries: {sql_validation['example_queries']['passed']}/{sql_validatio
n['example_queries']['passed'] + sql_validation['example_queries']['failed']}")

if total_failed > 0:
    print(f"\n❌ Found {total_failed} SQL syntax errors - review and fix before deploymen
t")
else:
    print(f"\n✅ All SQL syntax validation passed!")
```

Expected Output:

```
============================================================
VALIDATING SQL SYNTAX
============================================================

Validating SQL syntax...

Checking joins... ✅ 5/5
Checking measures... ✅ 18/18
Checking filters... ✅ 24/24
Checking dimensions... ✅ 15/15
Checking example queries... ✅ 10/10

======================================================================
SQL VALIDATION SUMMARY
======================================================================

📊 Overall: 72 passed, 0 failed
   • Joins: 5/5
   • Measures: 18/18
   • Filters: 24/24
   • Dimensions: 15/15
   • Example Queries: 10/10

✅ All SQL syntax validation passed!
```

Notebook 7: Genie Space Deployment

Cell 7.1: Create Genie Space

```
def create_genie_space(knowledge_store, workspace_client):
    """Create a new Genie space using Databricks SDK"""

    space_name = knowledge_store['space_name']
    warehouse_id = knowledge_store['warehouse_id']

    print(f"Creating Genie space: {space_name}")
    print(f"Using warehouse: {warehouse_id}")

    # Note: Genie API endpoints may vary - this is illustrative
    # In practice, use the actual Databricks SDK methods when available

    workspace_url = spark.conf.get("spark.databricks.workspaceUrl")
    token = dbutils.notebook.entry_point.getDbutils().notebook().getContext().apiToken().g
et()

    # Create space via REST API
    create_url = f"https://{workspace_url}/api/2.0/genie/spaces"
    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json"
    }

    payload = {
        "display_name": space_name,
        "description": knowledge_store['space_description'],
        "warehouse_id": warehouse_id
    }

    try:
        response = requests.post(create_url, headers=headers, json=payload)
        response.raise_for_status()

        space_data = response.json()
        space_id = space_data.get('id') or space_data.get('space_id')

        print(f"✅ Space created successfully")
        print(f"   Space ID: {space_id}")
        print(f"   Space URL: https://{workspace_url}/genie/spaces/{space_id}")

        return {
            'space_id': space_id,
            'space_name': space_name,
            'space_url': f"https://{workspace_url}/genie/spaces/{space_id}",
            'warehouse_id': warehouse_id,
            'status': 'created'
        }

    except requests.exceptions.RequestException as e:
        print(f"❌ Error creating space: {str(e)}")
        if hasattr(e.response, 'text'):
            print(f"   Response: {e.response.text}")
        raise
```

```
# Create Genie space
print("="*60)
print("CREATING GENIE SPACE")
print("="*60)
print()

space_info = create_genie_space(knowledge_store, w)

print(f"\n{'='*70}")
print("SPACE CREATION SUCCESSFUL")
print(f"{'='*70}")
print(f"Space ID: {space_info['space_id']}")
print(f"Space Name: {space_info['space_name']}")
print(f"Space URL: {space_info['space_url']}")
print(f"{'='*70}")
```

Expected Output:

```
============================================================
CREATING GENIE SPACE
============================================================

Creating Genie space: Sales Analytics Space
Using warehouse: abc123def456
✅ Space created successfully
   Space ID: 01ef5678-90ab-cdef-1234-567890abcdef
   Space URL: https://adb-123456789.12.azuredatabricks.net/genie/spaces/01ef5678-90ab-cdef
-1234-567890abcdef

======================================================================
SPACE CREATION SUCCESSFUL
======================================================================
Space ID: 01ef5678-90ab-cdef-1234-567890abcdef
Space Name: Sales Analytics Space
Space URL: https://adb-123456789.12.azuredatabricks.net/genie/spaces/01ef5678-90ab-cdef-12
34-567890abcdef
======================================================================
```

Cell 7.2: Add Tables to Space

```
def add_tables_to_space(space_id, knowledge_store):
    """Add Unity Catalog tables to Genie space with metadata"""

    workspace_url = spark.conf.get("spark.databricks.workspaceUrl")
    token = dbutils.notebook.entry_point.getDbutils().notebook().getContext().apiToken().g
et()

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json"
    }

    print(f"Adding {len(knowledge_store['tables'])} tables to space {space_id}...\n")

    tables_added = []

    for table_info in knowledge_store['tables']:
```

```
        full_name = table_info['full_name']

        # Add table to space
        add_table_url = f"https://{workspace_url}/api/2.0/genie/spaces/{space_id}/tables"

        payload = {
            "table_full_name": full_name,
            "description": table_info['description']
        }

        try:
            response = requests.post(add_table_url, headers=headers, json=payload)
            response.raise_for_status()

            print(f"✅ Added table: {full_name}")
            tables_added.append(full_name)

        except requests.exceptions.RequestException as e:
            print(f"⚠  Failed to add table {full_name}: {str(e)}")

    return {
        'space_id': space_id,
        'tables_added': tables_added,
        'total_tables': len(tables_added),
        'status': 'success' if len(tables_added) == len(knowledge_store['tables']) else 'p
artial'
    }

# Add tables
print("="*60)
print("ADDING TABLES TO SPACE")
print("="*60)
print()

table_result = add_tables_to_space(space_info['space_id'], knowledge_store)

print(f"\n{'='*70}")
print(f"TABLE ADDITION COMPLETE: {table_result['total_tables']}/{len(knowledge_store['tabl
es'])} tables added")
print(f"{'='*70}")
for table in table_result['tables_added']:
    print(f"   ✅ {table}")
print(f"{'='*70}")
```

Expected Output:

```
============================================================
ADDING TABLES TO SPACE
============================================================

Adding 5 tables to space 01ef5678-90ab-cdef-1234-567890abcdef...

✅ Added table: main.sales.orders
✅ Added table: main.sales.customers
✅ Added table: main.sales.products
✅ Added table: main.sales.order_items
✅ Added table: main.sales.warehouses
```

```
======================================================================
TABLE ADDITION COMPLETE: 5/5 tables added
======================================================================
   ✅ main.sales.orders
   ✅ main.sales.customers
   ✅ main.sales.products
   ✅ main.sales.order_items
   ✅ main.sales.warehouses
======================================================================
```

Cell 7.3: Configure Joins and Instructions

```
def configure_space_instructions(space_id, knowledge_store):
    """Add joins, SQL expressions, and global instructions to space"""

    workspace_url = spark.conf.get("spark.databricks.workspaceUrl")
    token = dbutils.notebook.entry_point.getDbutils().notebook().getContext().apiToken().g
et()

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json"
    }

    instructions_url = f"https://{workspace_url}/api/2.0/genie/spaces/{space_id}/instructi
ons"

    # Build comprehensive instructions document
    instructions_text = knowledge_store['global_instructions'] + "\n\n"

    # Add join instructions
    instructions_text += "=== TABLE RELATIONSHIPS ===\n\n"
    for join in knowledge_store['joins']:
        instructions_text += f"• {join['explanation']}\n"
        instructions_text += f"  SQL: {join['source_table']}.{join['source_column']} = {jo
in['target_table']}.{join['target_column']}\n\n"

    # Add common measures
    instructions_text += "\n=== AVAILABLE METRICS ===\n\n"
    for measure in knowledge_store['sql_expressions']['measures'][:10]:
        instructions_text += f"• {measure['name']}: {measure['description']}\n"

    # Add common filters
    instructions_text += "\n=== COMMON FILTERS ===\n\n"
    for filter_expr in knowledge_store['sql_expressions']['filters'][:10]:
        instructions_text += f"• {filter_expr['name']}: {filter_expr['description']}\n"

    payload = {
        "content": instructions_text
    }

    try:
        response = requests.post(instructions_url, headers=headers, json=payload)
        response.raise_for_status()

        print("✅ Instructions configured successfully")
```

```
        return {
            'space_id': space_id,
            'joins_configured': len(knowledge_store['joins']),
            'measures_documented': len(knowledge_store['sql_expressions']['measures']),
            'filters_documented': len(knowledge_store['sql_expressions']['filters']),
            'status': 'success'
        }

    except requests.exceptions.RequestException as e:
        print(f"❌ Error configuring instructions: {str(e)}")
        raise

# Configure instructions
print("="*60)
print("CONFIGURING SPACE INSTRUCTIONS")
print("="*60)
print()

instruction_result = configure_space_instructions(space_info['space_id'], knowledge_store)

print(f"\n{'='*70}")
print("INSTRUCTION CONFIGURATION COMPLETE")
print(f"{'='*70}")
print(f"   • Joins configured: {instruction_result['joins_configured']}")
print(f"   • Measures documented: {instruction_result['measures_documented']}")
print(f"   • Filters documented: {instruction_result['filters_documented']}")
print(f"{'='*70}")
```

Expected Output:

```
============================================================
CONFIGURING SPACE INSTRUCTIONS
============================================================

✅ Instructions configured successfully

======================================================================
INSTRUCTION CONFIGURATION COMPLETE
======================================================================
   • Joins configured: 5
   • Measures documented: 18
   • Filters documented: 24
======================================================================
```

Cell 7.4: Add Example Queries

```
def add_example_queries(space_id, knowledge_store):
    """Add example natural language queries to train Genie"""

    workspace_url = spark.conf.get("spark.databricks.workspaceUrl")
    token = dbutils.notebook.entry_point.getDbutils().notebook().getContext().apiToken().g
et()

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json"
    }
```

```
    print(f"Adding {len(knowledge_store['example_queries'])} example queries...\n")

    examples_added = 0

    for example in knowledge_store['example_queries']:
        # Create a conversation with the example query
        conversation_url = f"https://{workspace_url}/api/2.0/genie/spaces/{space_id}/conve
rsations"

        payload = {
            "content": example['natural_language']
        }

        try:
            response = requests.post(conversation_url, headers=headers, json=payload)
            response.raise_for_status()

            print(f"✅ Added: {example['natural_language'][:70]}...")
            examples_added += 1

        except requests.exceptions.RequestException as e:
            print(f"⚠  Failed to add example: {str(e)}")

    return {
        'space_id': space_id,
        'examples_added': examples_added,
        'status': 'success' if examples_added == len(knowledge_store['example_queries']) e
lse 'partial'
    }

# Add example queries
print("="*60)
print("ADDING EXAMPLE QUERIES")
print("="*60)
print()

example_result = add_example_queries(space_info['space_id'], knowledge_store)

print(f"\n{'='*70}")
print(f"EXAMPLE QUERIES ADDED: {example_result['examples_added']}/{len(knowledge_store['ex
ample_queries'])}")
print(f"{'='*70}")
```

Expected Output:

```
============================================================
ADDING EXAMPLE QUERIES
============================================================

Adding 10 example queries...

✅ Added: Show aggregated metrics from orders and customers...
✅ Added: Show aggregated metrics from order_items and orders...
✅ Added: Show aggregated metrics from order_items and products...
✅ Added: Calculate summary statistics for orders...
✅ Added: Combine data from customers and orders...
✅ Added: Combine data from orders and warehouses...
```

```
✅ Added: Query products table...
✅ Added: Combine data from customers and orders...
✅ Added: Show aggregated metrics from order_items and products...
✅ Added: Combine data from orders and customers...

======================================================================
EXAMPLE QUERIES ADDED: 10/10
======================================================================
```

Cell 7.5: Finalize and Test Deployment

```
def finalize_deployment(space_id, knowledge_store):
    """Finalize deployment and run validation tests"""

    workspace_url = spark.conf.get("spark.databricks.workspaceUrl")
    token = dbutils.notebook.entry_point.getDbutils().notebook().getContext().apiToken().g
et()

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json"
    }

    # Get space details
    space_url = f"https://{workspace_url}/api/2.0/genie/spaces/{space_id}"

    try:
        response = requests.get(space_url, headers=headers)
        response.raise_for_status()
        space_details = response.json()

        deployment_summary = {
            'space_id': space_id,
            'space_name': knowledge_store['space_name'],
            'space_url': f"https://{workspace_url}/genie/spaces/{space_id}",
            'deployment_time': datetime.now().isoformat(),
            'status': 'active',
            'configuration': {
                'tables': len(knowledge_store['tables']),
                'joins': len(knowledge_store['joins']),
                'measures': len(knowledge_store['sql_expressions']['measures']),
                'filters': len(knowledge_store['sql_expressions']['filters']),
                'dimensions': len(knowledge_store['sql_expressions']['dimensions']),
                'examples': len(knowledge_store['example_queries'])
            },
            'ready_for_use': True
        }

        # Save deployment summary
        output_file = f"{config['output_path']}/deployment_summary.json"
        with open(output_file.replace('/dbfs', ''), 'w') as f:
            json.dump(deployment_summary, f, indent=2)

        return deployment_summary

    except requests.exceptions.RequestException as e:
        print(f"❌ Error finalizing deployment: {str(e)}")
```

```
        raise

# Finalize deployment
print("="*60)
print("FINALIZING DEPLOYMENT")
print("="*60)
print()

deployment_summary = finalize_deployment(space_info['space_id'], knowledge_store)

print(f"\n{'='*80}")
print(f"{'🎉 DEPLOYMENT COMPLETE 🎉':^80}")
print(f"{'='*80}\n")

print(f"📊 DEPLOYMENT SUMMARY:")
print(f"   Space Name: {deployment_summary['space_name']}")
print(f"   Space ID: {deployment_summary['space_id']}")
print(f"   Deployment Time: {deployment_summary['deployment_time']}")
print(f"   Status: {deployment_summary['status'].upper()}")
print()
print(f"📈 CONFIGURATION:")
print(f"   • Tables: {deployment_summary['configuration']['tables']}")
print(f"   • Joins: {deployment_summary['configuration']['joins']}")
print(f"   • Measures: {deployment_summary['configuration']['measures']}")
print(f"   • Filters: {deployment_summary['configuration']['filters']}")
print(f"   • Dimensions: {deployment_summary['configuration']['dimensions']}")
print(f"   • Example Queries: {deployment_summary['configuration']['examples']}")
print()
print(f"{'='*80}")
print(f"🔗 ACCESS YOUR GENIE SPACE:")
print(f"{'='*80}")
print(f"{deployment_summary['space_url']}")
print(f"{'='*80}\n")

print(f"✅ The Genie space is ready for use!")
print(f"✅ Share the URL with your business users to start querying data in natural langua
ge")
print(f"\n💾 Deployment summary saved to: {config['output_path']}/deployment_summary.jso
n")
```

Expected Output:

```
============================================================
FINALIZING DEPLOYMENT
============================================================

================================================================================
                          🎉 DEPLOYMENT COMPLETE 🎉
================================================================================

📊 DEPLOYMENT SUMMARY:
   Space Name: Sales Analytics Space
   Space ID: 01ef5678-90ab-cdef-1234-567890abcdef
   Deployment Time: 2026-02-21T17:15:45.678901
   Status: ACTIVE

📈 CONFIGURATION:
   • Tables: 5
```

```
   • Joins: 5
   • Measures: 18
   • Filters: 24
   • Dimensions: 15
   • Example Queries: 10

================================================================================
🔗 ACCESS YOUR GENIE SPACE:
================================================================================
https://adb-123456789.12.azuredatabricks.net/genie/spaces/01ef5678-90ab-cdef-1234-567890ab
cdef
================================================================================

✅ The Genie space is ready for use!
✅ Share the URL with your business users to start querying data in natural language

💾 Deployment summary saved to: /dbfs/tmp/auto_genie_outputs/deployment_summary.json
```

Cell 7.6: Test the Deployed Space

```
def test_genie_space(space_id, test_questions):
    """Test the deployed Genie space with sample questions"""

    workspace_url = spark.conf.get("spark.databricks.workspaceUrl")
    token = dbutils.notebook.entry_point.getDbutils().notebook().getContext().apiToken().g
et()

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json"
    }

    test_results = []

    print("Testing Genie space with sample questions...\n")

    for question in test_questions:
        conversation_url = f"https://{workspace_url}/api/2.0/genie/spaces/{space_id}/conve
rsations"

        payload = {
            "content": question
        }

        try:
            response = requests.post(conversation_url, headers=headers, json=payload)
            response.raise_for_status()
            result = response.json()

            test_results.append({
                'question': question,
                'status': 'success',
                'response': result
            })

            print(f"✅ Question: {question}")
            print(f"   Status: SUCCESS\n")
```

```
        except requests.exceptions.RequestException as e:
            test_results.append({
                'question': question,
                'status': 'failed',
                'error': str(e)
            })
            print(f"❌ Question: {question}")
            print(f"   Status: FAILED - {str(e)}\n")

    return test_results

# Define test questions
test_questions = [
    "What was our total revenue last month?",
    "Show me the top 10 customers by order count",
    "How many orders were placed in each quarter of 2025?",
    "What are our best-selling products by revenue?"
]

print("="*60)
print("TESTING DEPLOYED GENIE SPACE")
print("="*60)
print()

test_results = test_genie_space(space_info['space_id'], test_questions)

print(f"\n{'='*70}")
print(f"TEST RESULTS: {len([r for r in test_results if r['status'] == 'success'])}/{len(te
st_results)} passed")
print(f"{'='*70}")

if all(r['status'] == 'success' for r in test_results):
    print("\n✅ All tests passed! The Genie space is functioning correctly.")
else:
    print("\n⚠  Some tests failed. Review the errors and adjust the knowledge store if nee
ded.")
```

Expected Output:

```
============================================================
TESTING DEPLOYED GENIE SPACE
============================================================

Testing Genie space with sample questions...

✅ Question: What was our total revenue last month?
   Status: SUCCESS

✅ Question: Show me the top 10 customers by order count
   Status: SUCCESS

✅ Question: How many orders were placed in each quarter of 2025?
   Status: SUCCESS

✅ Question: What are our best-selling products by revenue?
   Status: SUCCESS
```

```
======================================================================
TEST RESULTS: 4/4 passed
======================================================================

✅ All tests passed! The Genie space is functioning correctly.
```

Summary: Complete Notebook Flow

Notebook 1: Configuration & Setup → ✅ Connection validated

Notebook 2: Metadata Discovery → ✅ 5 tables, 40+ columns profiled

Notebook 3: Relationship Discovery → ✅ 5 relationships (confidence 0.75-1.0)

Notebook 4: Query Intelligence → ✅ 10 example queries from 2,847 historical queries

Notebook 5: Instruction Generation → ✅ Table docs, join explanations, 57 SQL expressions

Notebook 6: Assembly & Validation → ✅ Knowledge store validated (72/72 checks passed)

Notebook 7: Deployment → ✅ Genie space created, tested, and ready for use

Total Execution Time: ~15-30 minutes for 5 tables

Result: Production-ready Genie space accessible at generated URL

Would you like me to:

1. **Create a master orchestration notebook** that runs all 7 notebooks sequentially?
2. **Add error handling and retry logic** for robust production use?
3. **Build a configuration template** for easy customization?
4. **Create visualization dashboards** to monitor space usage post-deployment?